

Modelo de datos de Yalqui

Ochenta y siete tablas en doce módulos que siguen los dos ciclos de vida del producto. Yalqui no custodia dinero: registra el canon y cobra sus planes. Son dos flujos que el esquema mantiene separados por diseño.

Estado borrador, sin código Recaudo ninguno · solo trazabilidad Cliente propietario individual Reemplaza app_meta

Motor MySQL 8 · RDS Destino backend/src/db/schema.ts + backend/sql/schema.sql

00 Decisiones estructurales	01 Los dos flujos de dinero	02 El espinazo del modelo	03 Ciclos de vida → tablas
M1 Identidad y acceso	M2 La unidad y su canon	M3 Precio sugerido	M4 Demanda y aprobación
M5 Contrato digital	M6 Flujo A · arriendo	M7 Flujo B · Yalqui	M8 Operación e incidencias
M9 Riesgo y finanzas	M10 Perfil y scoring del inquilino	M11 Comunicación y vecindad	M12 Transversal
04 Convenciones	05 Fases de construcción	06 Lo que no se modela aún	07 Decisiones pendientes

00 Cinco decisiones estructurales

Todo lo demás se deriva de estas. Si alguna está mal, conviene corregirla ahora y no después de la primera migración.

Dos flujos de dinero, dos pilas de tablas

El arriendo (inquilino → propietario) y la monetización (propietario → Yalqui) tienen contraparte distinta, tratamiento tributario distinto, ciclo distinto y conciliación distinta. Y ahora también naturaleza distinta: el primero es un registro de algo que ocurrió por fuera, el segundo es plata que Yalqui sí cobra. Una sola tabla pagos obligaría a llaves foráneas nulas en la mitad de las filas y haría que «¿cuánto facturó Yalqui este mes?» dependa de un WHERE frágil. Se modelan como dos pilas paralelas: facturas_arriendo/pagos_arriendo y facturas_yalqui/pagos_yalqui.

Yalqui no recauda: registra

El canon se transfiere directo del inquilino al propietario. Yalqui puede poner el link de pago, pero el dinero nunca entra a una cuenta suya. Eso elimina la tabla `dispersiones`, elimina la figura de custodio de dinero de terceros — y cambia lo que `pagos_arriendo` significa: no es una transacción, es un **comprobante con estado de verificación**. La consecuencia práctica es que el registro puede quedar desactualizado frente a la realidad, así que el modelo necesita saber quién verificó cada pago y cuándo.

Los planes son datos, no un enum

El modelo de negocio es reciente y va a cambiar de nombres, precios y topes antes que el esquema. `planes` y `servicios` son tablas de catálogo con seeders: se siembran Básico, Profesional, Empresarial y Corporativo, y repreciar o renombrar es un `UPDATE`, no una migración.

El cliente es un propietario, no una inmobiliaria

El inmueble pertenece a un usuario y punto. No hay organizaciones, ni membresías, ni permisos delegados, ni un tercero administrando portafolio ajeno. Es la decisión que más tablas evita: sin ella habría que llevar `organizacion_id` en `inmuebles`, `contratos` y `suscripciones`, y resolver quién puede firmar en nombre de quién. **Es también la más cara de revertir**, porque toca la llave foránea más repetida del esquema — conviene tenerla presente si algún día el segmento cambia.

El rol no es una columna, y tampoco basta con una lista

Un propietario en Chapinero puede ser inquilino en Medellín, y un portero puede arrendar. Por eso `usuario_rol` es una tabla puente y no una columna. Pero con siete roles hace falta algo más: **saber sobre qué manda cada uno**. Un admin es global, un propietario manda sobre sus unidades, un socio sobre una sola, un portero sobre su edificio, un inquilino sobre su contrato. Sin ese alcance, «es propietario» no responde la única pregunta que importa — ¿de qué?

Los servicios a la carta cuelgan de un punto único

`servicios_contratados` es la bisagra entre el negocio y la operación: `valoraciones` (Pricing Engine) y `verificaciones` (Screening) la referencian de forma opcional. Así un servicio puede ejecutarse gratis dentro de un plan o cobrarse aparte, sin duplicar tablas.

Transparencia por diseño

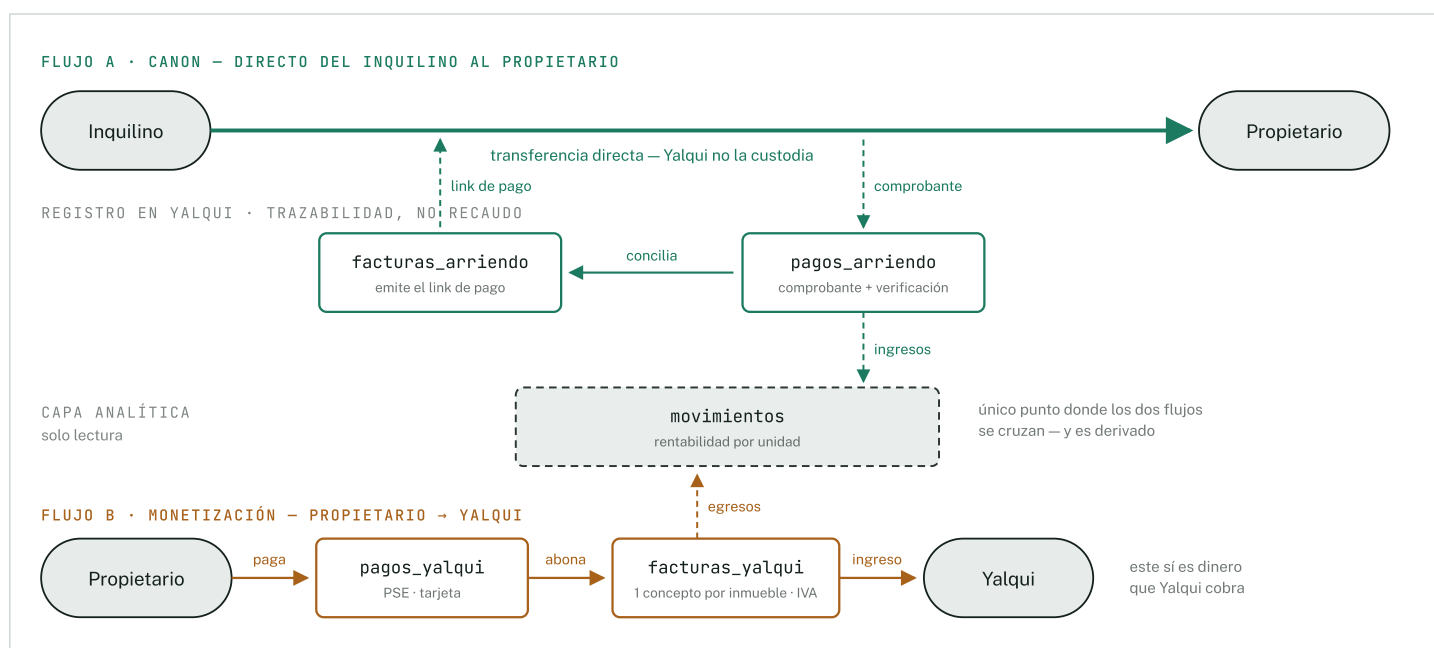
Si Yalqui aspira al estándar de Nubank, no basta con una interfaz limpia: **el esquema tiene que guardar la explicación, no solo el dato**. Ningún número que el usuario ve puede ser un resultado sin causa consultable. Por eso las facturas guardan conceptos y no totales, el score guarda eventos con su origen y no un puntaje suelto, y cada evento lleva además de su código interno una `explicacion_publica` escrita para una persona. La prueba de fuego: si el usuario pregunta «¿por qué este número?», la respuesta tiene que salir de una consulta, no de una llamada a soporte.

La rentabilidad es una capa derivada

movimientos es el único lugar donde los dos flujos se encuentran, y lo hace de solo lectura: se alimenta de los pagos **verificados** del arriendo y de las facturas de Yalqui para responder «¿cuánto me deja esta unidad?». No es transaccional, se puede reconstruir, y las proyecciones se calculan sobre ella — no se guardan.

01 Los dos flujos de dinero

Yalqui no cobra comisión sobre el canon, así que el dinero del arriendo y el dinero de Yalqui no comparten ni contraparte ni causa. El diagrama muestra dónde vive cada peso y dónde — y bajo qué condiciones — se cruzan.

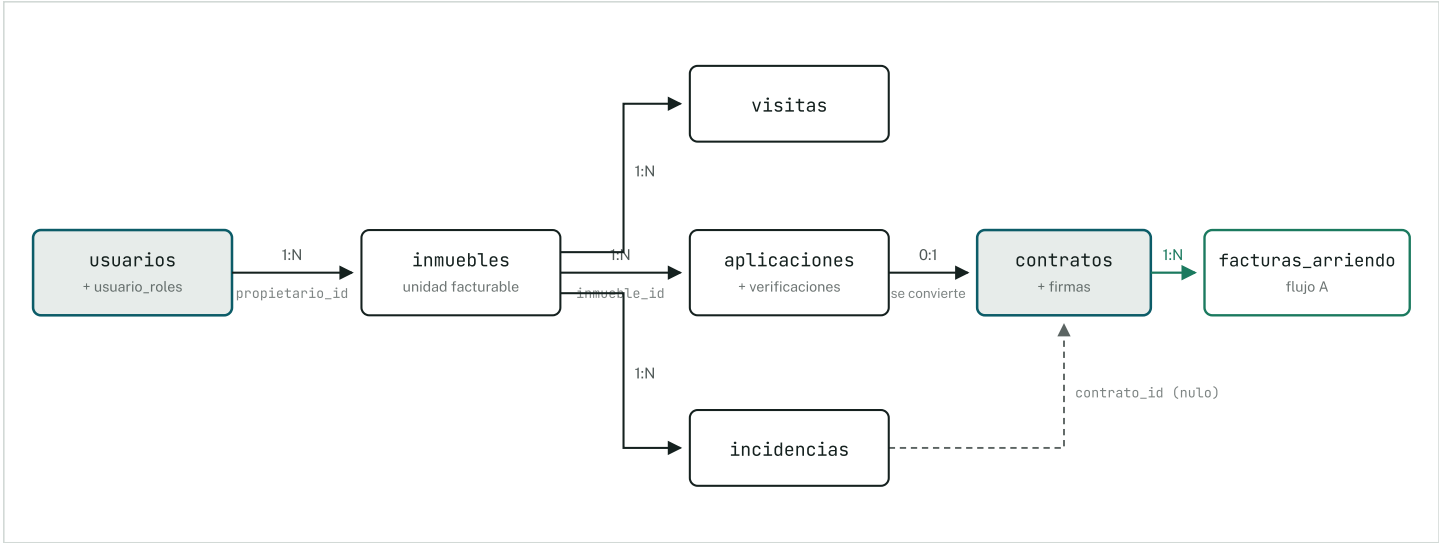


El único dinero que Yalqui cobra es el del flujo B. El canon viaja por fuera del sistema: Yalqui emite la factura, ofrece el link de pago y guarda el comprobante, pero nunca lo tiene en una cuenta suya. Por eso pagos_arriendo lleva estado de verificación y no estado de transacción, y por eso no existe dispersiones. Ninguna tabla del flujo A referencia una del flujo B.

Decidido: Yalqui no recauda. Se cae dispersiones y con ella la exposición a custodia de dinero de terceros. A cambio aparece un problema nuevo que el esquema tiene que resolver: **el registro puede mentir**. Un inquilino puede subir un comprobante falso o equivocado, o pagar sin subir nada. Por eso pagos_arriendo tiene un ciclo de verificación explícito — reportado → verificado o rechazado — y la mora se calcula sobre pagos **verificados**, nunca sobre pagos reportados.

02 El espinazo del modelo

Siete tablas sostienen el resto. Todo lo demás cuelga de alguna de ellas.



usuarios y **contratos** son los dos ejes. **usuarios** también aparece como **inquilino_id** en **aplicaciones** y **contratos**, y como **reportada_por_id** en **incidencias** — se omiten esas aristas para no volver ilegible el diagrama. **incidencias** cuelga del inmueble y no del contrato, porque un daño puede reportarse con el inmueble desocupado.

03 Los ciclos de vida, paso por paso

Cada paso documentado del producto escribe en tablas concretas. Si un paso no tiene tabla, no existe en el sistema; si una tabla no aparece aquí, sobra.

CICLO PROPIETARIO

PASO	TABLAS QUE ESCRIBE
Activación de cuenta	usuarios, usuario_rols, perfiles_propietario
Registro de la unidad	inmuebles (tipo, canon base), inmueble_etiquetas, inmueble_ajustes (amoblado, mascota, parqueadero...), cuentas_cobro, inmueble_fotos
Precio sugerido	valoraciones · si es de pago, también servicios_contratados
Precalificar en la visita	Manda el enlace desde la visita y sabe en minutos si le alcanza · precalificaciones
Promoción y visitas	inmuebles.estado → publicado, revisión de inmueble_fotos, publicaciones_externas en Facebook e Instagram, visitas
Aprobación de inquilino	aplicaciones, aplicacion_documentos, verificaciones, garantes · consulta scores_inquilino si ya arrendó antes
Contrato digital	plantillas_contrato según el tipo de unidad → contratos, contrato_ajustes (lo pactado), contrato_firmas con link por WhatsApp y OTP
Servicios públicos al día	Verificar lo que reportó el inquilino · facturas_cobro · alerta si hay recibo vencido
Comunicar a inquilinos	comunicados por unidad o portafolio → mensajes por WhatsApp, correo o app
Gestión operativa	facturas_arriendo, verificación de pagos_arriendo, mensajes de cobranza, incidencias, alertas
Control financiero	movimientos · proyecciones calculadas, no almacenadas
Renovación o cierre	negociaciones, contratos nuevo con contrato_anterior_id, o estado → terminado · inspecciones de salida

CICLO INQUILINO

PASO	TABLAS QUE ESCRIBE
Búsqueda de inmueble	solo lectura sobre inmuebles · sin escritura
Agendamiento de visita	visitas · el interesado puede ser anónimo todavía
Precalificación en la visita	Abre el enlace, llena sus datos e ingresos · precalificaciones, precalificacion_aportantes
Aplicación al arriendo	usuarios, perfiles_inquilino (ficha de caracterización), inquilino_atributos, consentimientos, aplicaciones, aplicacion_documentos
Verificación y aprobación	verificaciones con Truora y centrales · aplicacion_verificaciones deja constancia de cuáles respaldaron la decisión
Firma de contrato	Abre su link personal desde WhatsApp, verifica OTP y firma · contrato_firmas guarda IP, dispositivo, hora y hash
Pago mensual	transfiere por fuera y reporta en pagos_arriendo; el saldo de facturas_arriendo solo se mueve al verificar
Reporte de incidencias	incidencias, incidencia_eventos
Pedir ayuda a un vecino	solicitudes_ayuda → ayuda_respuestas · confirmar la ayuda recibida sube la convivencia de quien ayudó
Pagar lo que va aparte del canon	Ve su lista del mes — servicios y administración si le toca —, sube cada comprobante y lo marca · facturas_cobro
Comunicados recibidos	comunicado_destinatarios · confirma lectura cuando el aviso lo exige
Convivencia	lo miden pqrs de vecinos o administración e inspecciones periódicas
Renovación o salida	negociaciones o contrato_anexos tipo paz_y_salvo

M1 Identidad y acceso

Una sola tabla de personas. Los roles y los datos que cada rol necesita viven aparte, para que el mismo humano pueda arrendar y ser arrendatario sin duplicar cuenta.

Son **siete roles**, y cada uno manda sobre algo distinto. Dos de ellos se llaman «administrador» y no tienen nada que ver entre sí: el **de Yalqui** opera la plataforma con alcance global, el **del inmueble** administra una copropiedad y no ve una sola cifra de nadie. Esta matriz es lo que `permisos_rol` guarda como datos.

PUEDE...	ADMIN YALQUI	ADMIN INMUEBLE	PROPIETARIO	SOCIO	INQUILINO	PERSONAL	PROVEEDOR
Ver la ficha de la unidad	Todas	Las de su edificio, sin cifras	Las suyas	La suya	La que habita	Las de su edificio, sin cifras	—
Editar canon y ajustes	Sí	—	Sí	—	—	—	—
Publicar y despublicar	Sí	—	Sí	—	—	—	—
Aprobar o rechazar una aplicación	Sí	—	Sí	Solo si puede_decidir	—	—	—
Firmar el contrato	—	—	Sí	Si está en el título	Sí	—	—
Verificar un pago reportado	Sí	—	Sí	—	—	—	—
Ver rentabilidad y finanzas	Sí	Nunca	Total	Su porcentaje	—	Nunca	—
Reportar una incidencia de la unidad	Sí	Sí	Sí	Sí	Sí	Sí	—
Abrir una incidencia de áreas comunes	Sí	Sí	Sí	—	Sí	Sí	—
Cotizar y ejecutar mantenimiento	Sí	Contrata	Contrata	—	—	Interno	Sí
Registrar una PQRS	Sí	Sí	Sí	—	Sí	Sí	—
Resolver una PQRS	Sí	Sí, de su edificio	Sí, de su unidad	—	—	—	—
Confirmar que una visita ocurrió	Sí	Sí	Sí	—	—	Sí	—
Publicar un comunicado	Global	A su edificio	A sus unidades	—	—	—	—
Ver el score de un inquilino	Sí	Nunca	Con consentimiento	Solo el agregado	El suyo	Nunca	—
Cambiar de plan y pagarle a Yalqui	Sí	—	Sí	—	—	—	—

Los cuatro «Nunca» no son restricciones de conveniencia. El administrador del edificio y el portero necesitan saber que el 302 está ocupado y que hay una incidencia abierta; no necesitan saber cuánto paga quien vive ahí ni cómo se ha portado. Son los roles con más acceso físico y menos necesidad de dato sensible, y esa combinación se rompe sola si alguien reutiliza una pantalla de propietario sin pensarlo. El alcance de ambos es la edificación, y dentro de ella solo lo operativo.

Una consecuencia de dejar que el administrador resuelva PQRS: una queja resuelta_en_contra genera evento de score, y el score es portable. Es decir, el administrador de un edificio podría afectar la reputación de un inquilino **en toda la plataforma**, sin ser su arrendador y sin ser cliente de Yalqui. Por eso pqr gana confirmada_por_propietario: el administrador resuelve la queja para efectos de la convivencia del edificio, pero solo pesa en el score cuando el dueño de esa unidad lo confirma. Dos manos para una consecuencia que viaja — el mismo criterio que ya rige la ayuda entre vecinos.

Las dos filas con «Nunca» no son restricciones de conveniencia. Un portero necesita saber que el apartamento 302 está ocupado y que hay una incidencia abierta; no necesita saber cuánto paga quien vive ahí ni cómo se ha portado. Es el rol con más acceso físico y menos necesidad de dato sensible, y la combinación más peligrosa si se reutiliza una pantalla existente sin pensar. El alcance de persona_l_propiedad es la edificación, y dentro de ella solo lo operativo.

usuarios FASE 1

Toda persona con acceso al sistema, sin importar su rol.

CAMPOS	<code>id</code> , <code>uuid</code> , <code>email</code> UNIQUE , <code>password_hash</code> , <code>nombre</code> , <code>apellido</code> , <code>telefono</code> , <code>tipo_documento</code> enum CC·CE·NIT·PA, <code>numero_documento</code> , <code>estado</code> enum pendiente·activo·suspendido, <code>email_verificado_at</code> , <code>telefono_verificado_at</code> , <code>ultimo_acceso_at</code> , <code>created_at</code> , <code>updated_at</code>
ÍNDICES	UNIQUE (<code>tipo_documento</code> , <code>numero_documento</code>) · UNIQUE (<code>email</code>)
NOTAS	El <code>uuid</code> es el identificador público; el <code>id</code> nunca sale de la API. Sin columna <code>rol</code> — ver siguiente tabla.

usuario_roles FASE 1

Qué es cada usuario y sobre qué. Un usuario puede tener varios roles a la vez, y el mismo rol sobre varias cosas.

CAMPOS	<code>id</code> , <code>usuario_id</code> , <code>rol</code> enum <code>admin_yalqui</code> · <code>administrador_inmueble</code> · <code>propietario</code> · <code>socio_propietario</code> · <code>inquilino</code> · <code>personal_propiedad</code> · <code>proveedor</code> , <code>ambito_tipo</code> enum <code>global</code> · <code>inmueble</code> · <code>edificacion</code> · <code>contrato</code> , <code>ambito_id</code> UNSIGNED , <code>otorgado_at</code> , <code>otorgado_por_id</code> , <code>revocado_at</code>
ÍNDICES	UNIQUE (<code>usuario_id</code> , <code>rol</code> , <code>ambito_tipo</code> , <code>ambito_id</code>) — <code>ambito_id</code> = 0 para los globales, porque MySQL no aplica unicidad sobre nulos · IDX (<code>ambito_tipo</code> , <code>ambito_id</code>) — «quién manda sobre esto»
NOTAS	Solo <code>admin_yalqui</code> usa <code>ambito_tipo</code> = <code>global</code> . <code>propietario</code> y <code>socio_propietario</code> van por <code>inmueble</code> ; <code>administrador_inmueble</code> y <code>personal_propiedad</code> por <code>edificacion</code> ; <code>inquilino</code> por <code>contrato</code> . Revocar no borra: se sella <code>revocado_at</code> , porque hay que poder responder quién tenía qué permiso el día que se firmó un contrato.
CUIDADO	<code>administrador_inmueble</code> es el único rol que no se puede autootorgar ni conceder desde la app. A los demás los nombra alguien con interés directo: el propietario es dueño, el inquilino firmó, el portero lo contrató el edificio. Al administrador lo nombra la asamblea de copropietarios — gente que no es cliente de Yalqui — y con ese rol se le puede escribir a todo un edificio y abrir quejas contra sus residentes. Se otorga a mano, por un <code>admin_yalqui</code> , contra acta de asamblea o contrato de administración, y queda con <code>otorgado_por_id</code> y su rastro en auditoría.

etiquetas FASE 1

Cómo cada quien organiza su portafolio. Reemplazan la jerarquía que tenía inmuebles: un rótulo libre pesa menos y sirve para más.

CAMPOS	<code>id</code> , <code>ambito_tipo</code> enum <code>propietario</code> · <code>edificacion</code> , <code>ambito_id</code> , <code>nombre</code> VARCHAR (60), <code>color</code> CHAR (7), <code>orden</code> , <code>activo</code> , <code>creada_por_id</code> , <code>created_at</code>
ÍNDICES	UNIQUE (<code>ambito_tipo</code> , <code>ambito_id</code> , <code>nombre</code>) · IDX (<code>ambito_tipo</code> , <code>ambito_id</code> , <code>orden</code>)
NOTAS	El ámbito sigue el mismo patrón que <code>usuario_roles</code> : las del propietario son suyas y nadie más las ve; las de una edificación las administra el administrador del edificio. Un propietario etiqueta como le sirva — «Torre Almendros», «Herencia de papá», «Corto plazo», «Chapinero» — y esas categorías no tienen por qué coincidir con ninguna realidad física.

inmueble_etiquetas FASE 1

Qué etiquetas lleva cada unidad. Una unidad puede llevar varias.

CAMPOS	<code>inmueble_id</code> , <code>etiqueta_id</code> , <code>es_principal</code> BOOL , <code>asignada_at</code> , <code>asignada_por_id</code>
LLAVE	PRIMARY KEY (<code>inmueble_id</code> , <code>etiqueta_id</code>) · UNIQUE (<code>inmueble_id</code>) donde <code>es_principal</code> = 1
POR QUÉ ES_PRINCIPAL	Sin esa columna la vista por defecto miente. Si una unidad lleva «Chapinero» y «Herencia», al agrupar aparece en los dos grupos, y sumar el canon por grupo da más que el canon del portafolio. <code>es_principal</code> define en qué grupo cae la unidad cuando se agrupa, mientras las demás etiquetas siguen sirviendo para filtrar. Filtrar admite varias; agrupar exige una.
NOTAS	Las unidades sin etiqueta caen en un grupo «Sin etiqueta» que la interfaz muestra de último. No hay etiqueta obligatoria: forzarla convertiría el primer registro de una unidad en un trámite.

inmueble_propietarios FASE 1

Quiénes son dueños de una unidad y en qué proporción. Existe porque una unidad heredada entre tres hermanos, o comprada por una pareja, es lo normal y no la excepción.

CAMPOS	<code>id</code> , <code>inmueble_id</code> , <code>usuario_id</code> , rol enum principal·socio, porcentaje DECIMAL(5,2), aparece_en_titulo BOOL, puede_decidir BOOL, puede_ver_finanzas BOOL, desde DATE, hasta DATE NULL, created_at
ÍNDICES	UNIQUE(inmueble_id, usuario_id) donde hasta sea nulo · IDX(usuario_id)
NOTAS	<code>inmuebles.propietario_id</code> no desaparece: pasa a significar «el principal». Es quien administra la unidad en Yalqui, paga la suscripción y recibe el canon — y por eso sigue siendo la llave que usan suscripciones, contratos, facturas_yalqui y servicios_contratados. Los socios viven aquí. Tener dos lugares es deliberado: la alternativa era un JOIN en la consulta más frecuente del sistema para resolver algo que casi siempre tiene una sola respuesta.
CUIDADO	<code>aparece_en_titulo</code> no es cosmético: si la unidad tiene varios dueños registrados en el folio, el contrato lo firman todos o hay poder. Es la columna que decide a quién exige firma <code>contrato_firmas</code> . porcentaje alimenta el reporte de rentabilidad de cada socio — Yalqui informa la parte de cada uno, pero no la reparte: el canon sigue cayendo entero en la cuenta del principal.

personal_propiedad FASE 2

Porteros, vigilantes, aseo y mantenimiento interno. Gente atada a una **edificación**, no a una unidad — con acceso a lo operativo y nunca a lo financiero. El administrador del edificio **no** va aquí: es rol propio, porque decide y contrata mientras estos ejecutan.

CAMPOS	<code>id</code> , <code>usuario_id</code> NULL, <code>edificacion_id</code> , cargo enum portero·vigilante·aseo·jardineria·mantenimiento·otro, nombre, telefono, turno enum dia·noche·rotativo·administrativo, activo, desde DATE, hasta DATE NULL
ÍNDICES	IDX(edificacion_id, activo) · IDX(usuario_id)
NOTAS	No confundir con proveedores: el proveedor es externo y viene por un trabajo puntual con su cotización; el personal es del edificio y está siempre. Encaja en cosas que ya existían sin saber que lo esperaban — <code>pqrs.origen</code> ya contemplaba porteria y administracion, una visita la puede confirmar el portero, y en «Pedir ayuda a un vecino» es el que siempre está.
CUIDADO	Un portero nunca ve el canon ni el score de nadie. Es la regla más importante de este rol y la más fácil de romper por accidente al reutilizar una pantalla existente. Su alcance es la edificación, y dentro de ella solo lo operativo.

permisos_rol FASE 2

La matriz de abajo, como datos. Qué puede hacer cada rol en cada ámbito.

CAMPOS	<code>id</code> , rol enum, ambito_tipo enum, permiso VARCHAR(64), otorgado BOOL, condicion VARCHAR(64) NULL, nota
ÍNDICES	UNIQUE(rol, ambito_tipo, permiso)
NOTAS	<code>condicion</code> cubre los permisos que dependen de una columna y no solo del rol: un socio aprueba aplicaciones solo si <code>inmueble_propietarios.puede_decidir</code> está en verdadero, y un propietario ve el score de un aplicante solo si hay consentimiento vigente. Sin esa columna, esos casos terminan como if sueltos repartidos por el código, que es donde los permisos se rompen.

perfiles_propietario FASE 1

Datos fiscales y bancarios que solo importan cuando el usuario arrienda algo.

CAMPOS	<code>usuario_id</code> PK+FK, tipo_persona enum natural·juridica, razon_social, nit, digito_verificacion, responsable_iva, banco, tipo_cuenta enum ahorros·corriente, cuenta_token, cuenta_enmascarada, created_at, updated_at
NOTAS	El número de cuenta real no se guarda: se guarda el token de la pasarela más los últimos cuatro dígitos para mostrar en pantalla. <code>tipo_persona = juridica</code> significa una empresa dueña de su propio inmueble , no una administradora de inmuebles ajenos: eso último está fuera del segmento y fuera del modelo.

perfiles_inquilino FASE 1

Ficha de caracterización del inquilino: quién es, de qué vive y cómo prefiere pagar. Se llena una vez y se reutiliza en todas sus aplicaciones. Ver [M10](#) para el detalle completo y sus reglas de uso.

CAMPOS	<code>usuario_id</code> PK+FK, <code>fecha_nacimiento</code> DATE, <code>genero</code> enum femenino·masculino·no_binario·otro·prefiere_no_decir, <code>tipo_vinculacion</code> enum asalariado·independiente·estudiante·pensionado·desempleado·otro, <code>profesion</code> , <code>nivel_educativo</code> enum, <code>empresa</code> , <code>antiguedad_laboral_meses</code> , <code>rango_ingresos</code> enum en SMMLV, <code>ingresos_mensuales</code> DECIMAL(14,2) NULL, <code>dia_preferido_pago</code> TINYINT, <code>num_ocupantes_habitual</code> , <code>num_mascotas</code> TINYINT, <code>mascotas</code> JSON, <code>fumador</code> , <code>biografia</code> TEXT, <code>created_at</code> , <code>updated_at</code>
NOTAS	Sin columna edad: se guarda <code>fecha_nacimiento</code> y la edad se calcula, porque un número guardado envejece mal. Hobbies, habilidades e idiomas viven aparte en <code>inquilino_atributos</code> . El score no vive aquí — ver M10 .

proveedores FASE 2

Quien ejecuta mantenimientos. Puede ser una empresa sin cuenta de usuario todavía.

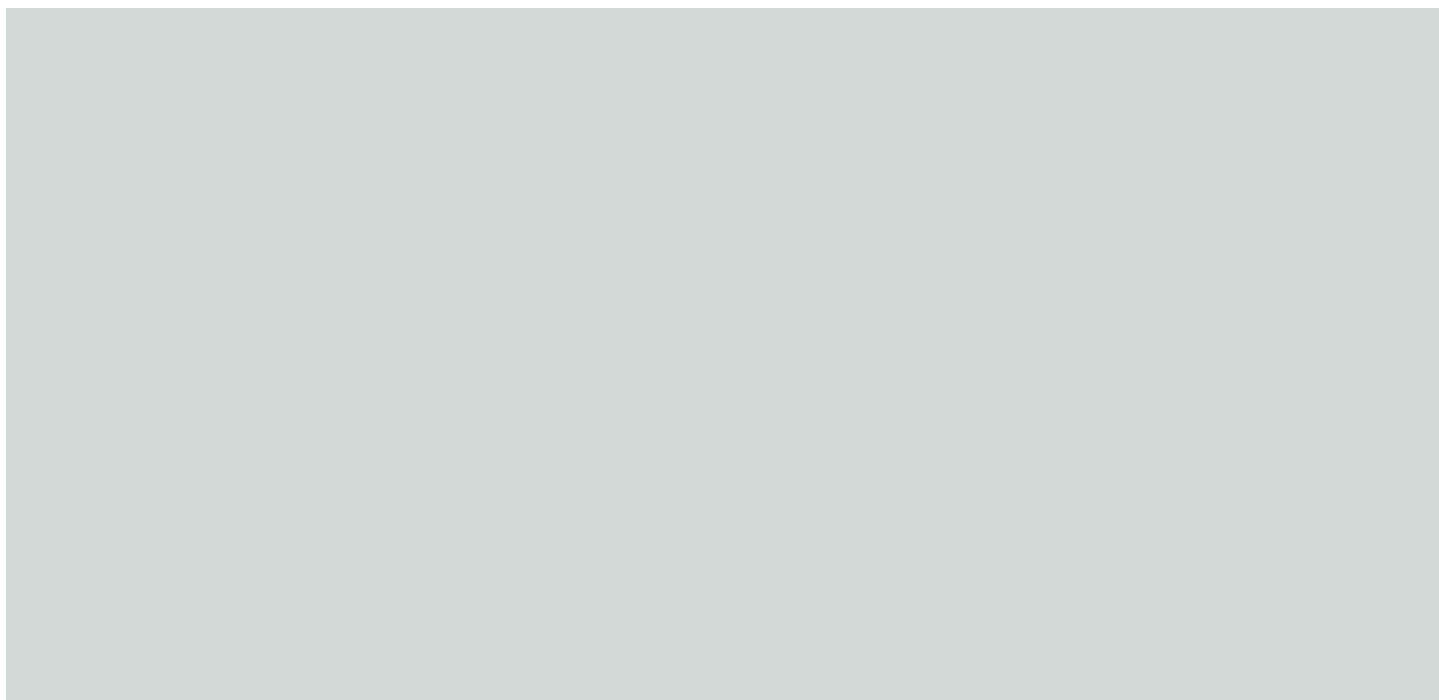
CAMPOS	<code>id</code> , <code>usuario_id</code> NULL, <code>razon_social</code> , <code>nit</code> , <code>especialidades</code> JSON, <code>ciudad</code> , <code>telefono</code> , <code>email</code> , <code>calificacion_promedio</code> DECIMAL(3,2), <code>trabajos_completados</code> , <code>activo</code> , <code>created_at</code>
RELACIONES	0:1 → <code>usuarios</code> · 1:N ← <code>incidencias</code>

M2 La unidad, su canon y sus servicios

Una fila por **unidad arrendable**: el apartamento, no el edificio. Es la unidad sobre la que se paga arriendo, la que se factura en el plan y sobre la que se mide rentabilidad. Un propietario tiene tantas filas como unidades arriende.

El módulo tiene tres partes: la **ficha** de la unidad, cómo se **arma su canon** a partir de un valor base más opciones configurables, y qué pasa con los **servicios públicos**.

La ficha



inmuebles FASE 1

Ficha de la unidad, su estado en el embudo y las condiciones con las que se publica.

`propietario_id` es el propietario principal; los socios van en `inmueble_propietarios`.

CAMPOS	<code>id</code> , <code>uuid</code> , <code>codigo_publico</code> , <code>propietario_id</code> , tipo <code>enum</code> <code>apartamento·casa·local·oficina·habitacion·parqueadero·bodega·lote</code> , estado <code>enum</code> <code>borrador·publicado·pausado·reservado·arrendado·archivado</code> , <code>direccion</code> , <code>complemento</code> , <code>barrio</code> , <code>ciudad</code> , <code>departamento</code> , <code>latitud DECIMAL(10,7)</code> , <code>longitud DECIMAL(10,7)</code> , <code>estrato</code> , <code>area_construida_m2</code> , <code>area_privada_m2</code> , <code>habitaciones</code> , <code>banos</code> , <code>parqueaderos</code> , <code>ocupantes_base TINYINT</code> , <code>ocupantes_maximo TINYINT</code> , <code>piso</code> , <code>anio_construccion</code> , <code>amoblado</code> , <code>mascotas_maximo TINYINT</code> , <code>mascotas_tipos_admitidos JSON</code> , <code>administracion_incluida</code> , <code>valor_administracion</code> , <code>canon_base DECIMAL(14,2)</code> , <code>deposito</code> , <code>tope_ingreso_pct DECIMAL(5,2) = 50.00</code> , <code>servicios_publicos_incluidos enum ninguno·algunos·todos</code> , <code>matricula_inmobiliaria</code> , <code>chip_catastral</code> , <code>descripcion</code> <code>TEXT</code> , <code>edificacion_id NULL</code> , <code>publicado_at</code> , <code>created_at</code> , <code>updated_at</code>
ÍNDICES	<code>IDX(propietario_id) · IDX(ciudad, estado) · IDX(estado, tipo, canon_base) · IDX(edificacion_id)</code>
NOTAS	<code>propietario_id</code> es el propietario principal : quien administra la unidad en Yalqui, paga la suscripción y recibe el canon. Los copropietarios viven en <code>inmueble_propietarios</code> con su porcentaje. <code>matricula_inmobiliaria</code> y <code>chip_catastral</code> quedan desde ya: son la llave de cruce con notariado y catastro cuando lleguen esas integraciones. Se cayó <code>inmueble_padre_id</code> . Una unidad ya no pertenece a otra: el portafolio es plano y se organiza con etiquetas , que son más flexibles que una jerarquía. Un <code>parqueadero</code> puede ser unidad arrendable por sí sola — con su propio contrato y su propia suscripción — o un accesorio del apartamento, y entonces no es una fila aquí sino un ajuste al canon. <code>canon_base</code> se llama así y no <code>canon_publicado</code> porque casi nunca es lo que el inquilino termina pagando: es el punto de partida al que se le suman los ajustes.
CUÁNTA GENTE Y CUÁNTAS MASCOTAS CABEN	<code>ocupantes_base</code> es cuántas personas incluye el canon base y <code>ocupantes_maximo</code> el tope del inmueble. Faltaban, y sin ellos el ajuste de «persona adicional» no tiene contra qué contarse: adicional a qué .
MASCOTAS, CON NÚMERO	<code>admite_mascotas</code> era un booleano y no alcanzaba: no puede decir «hasta dos», que es la respuesta real de casi todo propietario. Pasa a <code>mascotas_maximo</code> , donde 0 significa que no admite, y a <code>mascotas_tipos_admitidos</code> para el caso frecuente de «perro pequeño sí, perro grande no». Lo mismo del otro lado: el inquilino declara <code>num_mascotas</code> y no un sí o no. Y las dos cosas juntas permiten al buscador filtrar por «somos cuatro con dos perros», que es como la gente busca de verdad y hoy no se podía.
EL TOPE	<code>tope_ingreso_pct</code> es la regla de precalificación de esta unidad: por defecto 50%, y el propietario puede ser más exigente. Vive en la unidad y no en el código porque es una decisión de negocio que va a cambiar, y porque no tiene por qué ser la misma para un apartaestudio de \$900.000 que para un apartamento de \$6.000.000 — a ingresos altos, gastar la mitad en arriendo pesa distinto.
OTO	La jerarquía sí expresaba algo que una etiqueta no: que ese <code>parqueadero</code> no se puede arrendar aparte . Con etiquetas, dos unidades solo quedan co-rotuladas. Si esa restricción importa — vender el apartamento con el <code>parqueadero</code> obligatorio, por ejemplo — habría que expresarla de otra forma; hoy el modelo no la tiene.

edificaciones FASE 2

La edificación física: el edificio, el conjunto o la zona. Define quién es vecino de quién, quién manda ahí, y — lo que faltaba — de quién son las zonas comunes y quién paga sus gastos.

CAMPOS	<code>id</code> , nombre, tipo enum edificio·conjunto·casa_dividida·zona, regimen enum copropiedad·propiedad_unica·informal, <code>propietario_id</code> NULL, direccion, barrio, ciudad, latitud DECIMAL(10,7), longitud DECIMAL(10,7), num_unidades, area_comun_m2, administracion_nombre, administracion_telefono, activo, created_at
RELACIONES	1:N → inmuebles vía edificacion_id nullable · 1:N → movimientos de ámbito edificación · 1:N → personal_propiedad · 1:N → pqrs
LOS DOS REGÍMENES	Este es el campo que faltaba. Hay dos situaciones que se ven parecidas y no lo son: <code>copropiedad</code> — muchos dueños, asamblea, administrador nombrado por ella. Las zonas comunes son de todos y el propietario de Yalqui tiene una unidad ahí. Él no decide sobre el edificio y paga administración. <code>propiedad_unica</code> — un solo dueño de toda la edificación, que arrienda las unidades por separado: el edificio de apartaestudios, la casa dividida en habitaciones, la casa con local abajo. Las zonas comunes son suyas y los gastos de ellas también. Pintar la fachada lo paga él. Cuando regimen = <code>propiedad_unica</code>, <code>propietario_id</code> apunta a su dueño y ese usuario tiene autoridad de administrador sobre la edificación sin otorgamiento manual: es suya. En <code>copropiedad</code> el rol lo sigue otorgando a mano un admin de Yalqui contra acta de asamblea, porque ahí la autoridad viene de gente que no es cliente.
NO ES UNA ETIQUETA	La edificación es un hecho verificable: existe, tiene dirección, tiene zonas comunes que cuestan plata. Una etiqueta es cómo alguien organizó su lista. No se le puede dar autoridad a nadie sobre un rótulo escrito a mano, ni decidir quién es vecino por coincidencia de texto, ni cargarle un gasto de fachada. Por eso Pedir Ayuda a Vecino , <code>personal_propiedad</code> , las PQRS, las incidencias de áreas comunes y los gastos de edificación cuelgan de <code>edificacion_id</code> y nunca de una etiqueta.
SIEMPRE OPCIONAL	<code>inmuebles.edificacion_id</code> es nulo y así se queda. Los dos caminos de registro son de primera clase: crear la edificación primero y colgarle unidades — lo natural para el dueño de un edificio de apartaestudios —, o registrar una unidad suelta sin edificación ninguna, que es el caso del propietario con un apartamento en cualquier parte. Ninguno de los dos es el camino «correcto» del que el otro es la excepción.
QUÉ SE PIERDE SIN EDIFICACIÓN	Una unidad suelta no tiene zonas comunes, ni vecinos verificados, ni administrador, ni gastos de edificación. No es una carencia: es que esas cosas no existen para ella. Si más adelante resulta que sí pertenece a un edificio, asociarla es llenar una columna nula — no hay migración ni pérdida de historia.

inmueble_fotos FASE 1

Galería ordenada del inmueble. El archivo en sí vive en archivos.

CAMPOS	<code>id</code> , <code>inmueble_id</code> , <code>archivo_id</code> , orden, es_portada, descripcion, ancho, alto, bytes, hash_percep CHAR(16), estado_revision enum pendiente·apta·con_observaciones·rechazada, observaciones JSON, revisada_at
ÍNDICES	IDX(inmueble_id, orden) · IDX(inmueble_id, estado_revision)
NOTAS	Las dimensiones y el hash_percep se calculan al subir, no al promocionar: así el botón de promocionar responde al instante en vez de procesar diez imágenes. El hash perceptual detecta la misma foto subida dos veces aunque cambie de tamaño o formato — el error más común al armar una galería. observaciones guarda qué falló y por qué, en lenguaje que el propietario entienda.

amenidades · inmueble_amenidades FASE 3

Catálogo de características filtrables (gimnasio, ascensor, portería 24h) y su tabla puente.

CAMPOS	amenidades: <code>codigo</code> PK, nombre, categoria, icono, activo · puente: PK(inmueble_id, amenidad_codigo)
NOTAS	Hasta la fase 3 basta con un campo JSON en inmuebles; se normaliza cuando el buscador necesite filtrar por ellas.

Cómo se arma el canon

Una unidad no tiene un precio, tiene un precio **base** y un conjunto de opciones que lo mueven: amoblarla, permitir mascota, sumar un parqueadero, aceptar un ocupante más, incluir el aseo. Cada opción tiene su propio valor y el propietario decide cuáles ofrece y a cuánto.

Se modela con tres tablas y una regla: el catálogo dice **qué ajustes existen**, la unidad dice **cuáles ofrece y a qué precio**, y el contrato congela **cuáles se pactaron**. Ese último es el que la factura desglosa mes a mes.

UN CANON DE EJEMPLO	DE DÓNDE SALE	VALOR
Canon base	<code>inmuebles.canon_base</code>	\$1.800.000
Amoblado	<code>contrato_ajustes · amoblado</code>	+ \$250.000
Parqueadero cubierto	<code>contrato_ajustes · parqueadero × 1</code>	+ \$150.000
Mascota	<code>contrato_ajustes · mascota × 1</code>	+ \$80.000
Persona adicional	<code>contrato_ajustes · persona_adicional × 2</code>	+ \$120.000
Administración	<code>inmuebles.valor_administracion</code>	+ \$320.000
Total mensual	seis líneas en <code>factura_arriendo_conceptos</code>	\$2.720.000

POR QUÉ ASÍ El inquilino no recibe «arriendo: \$2.720.000». Recibe las seis líneas, cada una con su nombre. Es el principio de transparencia aplicado donde más importa, y de paso hace que negociar sea posible: se puede quitar el parqueadero sin renegociar el canon entero.

catalogo_ajustes

FASE 1

Qué tipos de ajuste existen en la plataforma. Configurable: agregar «depósito adicional» o «servicio de aseo» no requiere tocar código.

CAMPOS

`id`, código **UNIQUE** `amoblado · persona_adicional · mascota · parqueadero · deposito · aseo · internet · servicios_publicos`, `nombre`, `descripcion`, `categoria` **enum** `comodidad · ocupacion · mascota · parqueadero · servicio · otro`, `tipo_calculo` **enum** `monto_fijo · porcentaje · por_cantidad`, `periodicidad` **enum** `mensual · unico`, `permite_cantidad` **BOOL**, `aplica_a_tipos` **JSON**, `icono`, `activo`, `orden`

NOTAS

`tipo_calculo` cubre los tres casos reales: amoblado es monto fijo, persona adicional es por cantidad, y algún día alguien va a querer «mascota: +5% del canon». `aplica_a_tipos` evita ofrecer «persona adicional» en un parqueadero. El catálogo lo administra Yalqui, no el propietario — así los avisos son comparables entre sí y el buscador puede filtrar por «admite mascota» sin adivinar cómo lo escribió cada quien.

inmueble_ajustes

FASE 1

Qué ofrece esta unidad y a cuánto. Es la lista de precios del propietario para su unidad.

CAMPOS

`id`, `inmueble_id`, `ajuste_id`, `disponible` **BOOL**, `valor` **DECIMAL**(14,2), `porcentaje` **DECIMAL**(6,3), `cantidad_maxima`, `obligatorio` **BOOL**, `nota`, `created_at`, `updated_at`

ÍNDICES

UNIQUE(`inmueble_id`, `ajuste_id`)

NOTAS

`obligatorio` es para lo que no es opcional aunque sea un ajuste: un edificio que exige comprar el parqueadero con el apartamento. `cantidad_maxima` pone el tope — dos personas adicionales, una mascota. Y `disponible = false` con un valor guardado permite pausar una opción sin perder el precio que ya tenía configurado.

Promoción en canales externos

Un botón por unidad que revisa las fotos y la publica afuera. Antes de modelarlo hay que decir algo incómodo: **Facebook Marketplace no tiene API pública para publicar arriendos**. Existe un [programa de socios cerrado](#) con [API de items](#), y en vivienda opera a través de portales ya aprobados como Apartments.com o Zumper, todos en Estados Unidos. Para una plataforma nueva en Colombia, no es una vía realista hoy.

Lo que sí se puede automatizar con Meta, sin permisos especiales: publicar en una **página de Facebook**, publicar en **Instagram** con cuenta de empresa, y alimentar un **catálogo de tipo home_listing** que sirve para anuncios pagos de inmuebles. El modelo trata los cuatro como canales; el de Marketplace existe en el catálogo pero nace deshabilitado.

canales_publicacion FASE 2

Catálogo de dónde se puede publicar y bajo qué condiciones. Es lo que permite prender un canal el día que se consiga el permiso, sin tocar código.

CAMPOS	<code>id</code> , <code>codigo</code> UNIQUE <code>facebook_page</code> · <code>instagram</code> · <code>meta_catalog</code> · <code>marketplace_partner</code> · <code>portal_externo</code> , <code>nombre</code> , <code>proveedor</code> , <code>requiere_aprobacion</code> BOOL, <code>estado</code> enum <code>disponible</code> · <code>no_disponible</code> · <code>en_gestion</code> , <code>países</code> JSON, <code>costo_por_publicacion</code> DECIMAL(10,2), <code>limite_fotos</code> , <code>requisitos</code> JSON, <code>activo</code>
NOTAS	<code>requisitos</code> guarda lo que cada canal exige — mínimo de fotos, resolución, largo del texto — y es contra eso que corre la revisión previa. <code>marketplace_partner</code> se siembra con <code>estado = no_disponible</code> : la fila existe para que el día que Meta apruebe una alianza sea un UPDATE y no un rediseño.

publicaciones_externas FASE 2

Una unidad publicada en un canal, con su identificador allá afuera. Es lo que permite despublicar cuando se arrienda.

CAMPOS	<code>id</code> , <code>inmueble_id</code> , <code>canal_id</code> , <code>publicada_por_id</code> , <code>estado</code> enum <code>borrador</code> · <code>en_revision</code> · <code>publicada</code> · <code>rechazada</code> · <code>pausada</code> · <code>expirada</code> · <code>eliminada</code> , <code>id_externo</code> , <code>url_externa</code> , <code>titulo</code> , <code>cuerpo</code> TEXT, <code>fotos_incluidas</code> JSON, <code>respuesta</code> JSON, <code>motivo_rechazo</code> , <code>publicada_at</code> , <code>expira_at</code> , <code>ultima_sync_at</code>
ÍNDICES	UNIQUE(<code>canal_id</code> , <code>id_externo</code>) · IDX(<code>inmueble_id</code> , <code>estado</code>)
NOTAS	Publicar es fácil; despublicar es lo que se olvida. Cuando <code>inmuebles.estado</code> pasa a <code>arrendado</code> , hay que bajar el aviso de todos los canales — sin <code>id_externo</code> guardado eso es imposible y el propietario queda recibiendo llamadas por un inmueble que ya arrendó. <code>fotos_incluidas</code> congela qué fotos se mandaron, porque la galería cambia después.

publicacion_metricas FASE 3

Si el canal sirve o no. Sin esto, promocionar es fe.

CAMPOS	<code>id</code> , <code>publicacion_id</code> , <code>fecha</code> DATE, <code>impresiones</code> , <code>clics</code> , <code>mensajes</code> , <code>guardados</code> , <code>costo</code> DECIMAL(10,2), <code>sincronizado_at</code>
ÍNDICES	UNIQUE(<code>publicacion_id</code> , <code>fecha</code>)
NOTAS	Cruzado con <code>visitas</code> y <code>aplicaciones</code> responde la pregunta que de verdad importa: cuánto cuesta conseguir un inquilino por cada canal. Eso es lo que después justifica cobrar la promoción como servicio a la carta.

Servicios públicos

Que el propietario pueda ver si el inquilino ya pagó la energía es, en la práctica, una alerta de mora temprana: el que deja vencer el recibo de servicios suele estar a un mes

de dejar vencer el canon. Y saber si están incluidos o no es información que el aviso necesita desde el primer día.

Se separa en dos niveles: la **cuenta** con cada empresa, que es estable y va por unidad, y las **facturas** de cada periodo, que son las que se consultan por API. La columna **servicios_publicos_incluidos** en inmuebles es el resumen para el buscador; el detalle real está aquí.

empresas_servicio_publico FASE 2

Catálogo de prestadores de servicios públicos. En Colombia son regionales, así que la lista depende de la ciudad de la unidad. La administración no pasa por acá: su contraparte es la copropiedad.

CAMPOS	<code>id</code> , <code>codigo</code> UNIQUE, <code>nombre</code> , <code>tipo_servicio</code> enum <code>energia·acueducto·gas·aseo·internet·tv·telefonía</code> , <code>ciudades</code> JSON, <code>tiene_api</code> BOOL, <code>metodo_consulta</code> enum <code>api·scraping·comprobante·ninguno</code> , <code>url_portal</code> , <code>activo</code>
NOTAS	<code>metodo_consulta</code> es honestidad de diseño: muy pocas empresas de servicios públicos en Colombia exponen una API abierta. La mayoría va a arrancar en <code>comprobante</code> — el inquilino sube el recibo pagado, igual que hace con el canon — y algunas se podrán automatizar después. El esquema es el mismo en los tres casos; lo único que cambia es la columna <code>fuelle</code> de la factura.

cuentas_cobro FASE 2

Todo lo que se paga **aparte del canon** mes a mes: los servicios públicos y, cuando aplica, la administración. Es donde queda registrado quién responde por cada uno y si va incluido en el canon.

CAMPOS	<code>id</code> , <code>inmueble_id</code> , <code>tipo</code> enum <code>energia·acueducto·gas·aseo·internet·tv·telefonía·administración·cuota_extraordinaria</code> , <code>empresa_id</code> NULL, <code>edificacion_id</code> NULL, <code>numero_cuenta</code> , <code>titular</code> enum <code>propietario·inquilino·administración</code> , <code>responsable_pago</code> enum <code>propietario·inquilino</code> , <code>incluido_en_canon</code> BOOL, <code>promedio_mensual</code> DECIMAL(14,2), <code>tope_incluido</code> DECIMAL(14,2) NULL, <code>dia_vencimiento</code> TINYINT, <code>activo</code> , <code>created_at</code> , <code>updated_at</code>
ÍNDICES	UNIQUE(<code>inmueble_id</code> , <code>tipo</code> , <code>numero_cuenta</code>) · IDX(<code>inmueble_id</code> , <code>activo</code> , <code>responsable_pago</code>)
CUÁNDO APLICA LA ADMINISTRACIÓN	Aparece como cuenta aparte solo si se cumplen las dos cosas : la unidad está en una edificación de régimen copropiedad, y <code>incluido_en_canon</code> es falso. Si va incluida, no es una cuenta del inquilino sino una línea de su factura de arriendo, y él no tiene que hacer nada. Si la edificación es de <code>propiedad_unica</code> , no hay administración que pagar: no existe copropiedad a la cual pagarle. Por eso <code>empresa_id</code> pasa a ser nulo y aparece <code>edificacion_id</code> — la contraparte de la administración es la copropiedad, no una empresa de servicios.
CUOTA EXTRAORDINARIA	Va aparte de la ordinaria a propósito. En Colombia la cuota extraordinaria — el arreglo de la fachada, el cambio del ascensor — suele ser obligación del propietario y no del inquilino, aunque la ordinaria la pague el inquilino. Con un solo tipo «administración» esa distinción se pierde y termina en discusión. Por defecto nace con <code>responsable_pago = propietario</code> .
NOTAS	<code>titular</code> y <code>responsable_pago</code> no son lo mismo y confundirlos es un problema real: la cuenta suele quedar a nombre del propietario aunque pague el inquilino, y eso define a quién le llega el corte. <code>tope_incluido</code> cubre el caso frecuente de «agua incluida hasta \$80.000, el excedente lo paga el inquilino» — sin esa columna, ese acuerdo se vuelve una discusión mensual. Cuando <code>incluido_en_canon</code> es verdadero, el gasto entra como egreso del propietario en <code>movimientos</code> .

facturas_cobro

FASE 3

El recibo de un periodo y su estado de pago — de un servicio o de la administración. Lo ve el propietario para saber si el inquilino está al día, y lo ve el inquilino como su lista mensual de lo que paga aparte del arriendo.

CAMPOS	<code>id</code> , <code>cuenta_id</code> , <code>contrato_id</code> NULL, <code>periodo</code> CHAR(7), <code>fecha_emision</code> , <code>fecha_vencimiento</code> , <code>valor</code> DECIMAL(14,2), <code>consumo</code> , <code>unidad_consumo</code> , <code>estado</code> enum pendiente·reportada·verificada·vencida·en_acuerdo·desconocido, <code>quien_reporta</code> enum inquilino·propietario·api, <code>reportado_por_id</code> NULL, <code>comprobante_archivo_id</code> NULL, <code>fecha_pago_declarada</code> DATE, <code>reportada_at</code> , <code>verificado_por_id</code> NULL, <code>verificada_at</code> , <code>motivo_rechazo</code> , <code>fuentes</code> enum api·comprobante·manual, <code>sincronizado_at</code> , <code>respuesta</code> JSON, <code>created_at</code>
ÍNDICES	UNIQUE(<code>cuenta_id</code> , <code>periodo</code>) · IDX(<code>estado</code> , <code>fecha_vencimiento</code>) · IDX(<code>contrato_id</code> , <code>periodo</code>) — la lista mensual del inquilino
EL INQUILINO LOS SUBE	Cada mes el inquilino ve su lista: los servicios que le tocan según <code>cuentas_cobro.responsable_pago</code> , con lo que ya se sabe del recibo. Sube el comprobante, declara la fecha en que pagó y lo marca. Queda en reportada, no en pagada — el mismo ciclo de dos manos que el canon: alguien reporta, alguien verifica. Si la empresa tiene API, el estado salta a verificada solo; si no, lo confirma el propietario.
NOTAS	Solo aparecen en su lista los cobros con <code>responsable_pago = inquilino</code> ; los que van incluidos en el canon no le corresponden y no los ve como pendientes. La administración sigue la misma regla que la energía: si le toca, la sube y la marca; si va en el canon, ni aparece. <code>desconocido</code> es un estado de primera clase y no un NULL: si la empresa no tiene API y nadie subió el recibo, ambos ven «no sabemos», no «pendiente» — decirle a alguien que tiene una deuda que nadie verificó es peor que no decirle nada. Guardar <code>consumo</code> permite ver la fuga de agua antes que la factura impagada.
Y EL SCORE	Un recibo vencido dispara alerta al propietario, pero no genera evento de score: es deuda con un tercero, no con él. Ahora bien, cumplir sí podría sumar — un inquilino que sube sus seis comprobantes a tiempo todos los meses está demostrando algo. Lo dejo anotado como candidato a la dimensión <code>documentacion</code> , que hoy tiene peso cero.

M3

Precio sugerido · Pricing Engine

valoraciones

FASE 2

Cada corrida del motor de precios sobre un inmueble. Tabla de solo inserción: nunca se actualiza una valoración, se genera otra.

CAMPOS	<code>id</code> , <code>inmueble_id</code> , <code>servicio_contratado_id</code> NULL, <code>canon_sugerido</code> DECIMAL(14,2), <code>rango_min</code> , <code>rango_max</code> , <code>ocupantes_base</code> TINYINT, <code>costo_servicios_incluidos</code> DECIMAL(14,2), <code>confianza</code> DECIMAL(4,3), <code>version_modelo</code> , <code>comparables</code> JSON, <code>supuestos</code> JSON, <code>generada_at</code> , <code>aplicada</code> , <code>aplicada_at</code>
RELACIONES	N:1 → <code>inmuebles</code> · 0:1 → <code>servicios_contratados</code>
SUGIERE LA ESTRUCTURA, NO UN NÚMERO	Acá había una inconsistencia. El canon de una unidad es base más ajustes — amoblado, parqueadero, mascota, persona adicional — pero el motor sugería un solo número, como si el precio fuera plano. Por eso aparece <code>valoracion_ajustes</code> : el motor propone el canon base y cuánto cobrar por cada opción. Es la misma lógica del canon configurable, aplicada al lado que la estima.
EL NÚMERO DE PERSONAS	<code>ocupantes_base</code> entra como supuesto explícito del avalúo: un apartaestudio que sirve para dos no vale lo mismo que uno idéntico donde caben cuatro, y hoy el motor no lo sabía. Los comparables solo son comparables a igual capacidad.
NOTAS	Guardar <code>version_modelo</code> y <code>comparables</code> permite explicarle al propietario por qué se sugirió ese precio meses después, y auditar el motor cuando cambie.

valoracion_ajustes FASE 2

Cuánto sugiere cobrar el motor por cada opción, no solo por el inmueble pelado.

CAMPOS	<code>id</code> , <code>valoracion_id</code> , <code>ajuste_id</code> , <code>valor_sugerido</code> DECIMAL(14,2), <code>rango_min</code> , <code>rango_max</code> , <code>base_calculo</code> enum <code>comparables·costo_marginal·mixto</code> , <code>justificacion</code> , <code>confianza</code> DECIMAL(4,3)
ÍNDICES	UNIQUE(<code>valoracion_id</code> , <code>ajuste_id</code>)
PERSONA ADICIONAL, Y EL PISO QUE NADIE CALCULA	Acá está el caso que motivó todo esto. Si la unidad tiene servicios incluidos en el canon , cada ocupante de más le cuesta plata real al propietario: más agua, más energía, más gas. El motor calcula ese costo marginal dividiendo <code>cuentas_cobro.promedio_mensual</code> entre los ocupantes y propone un piso. Cobrar \$60.000 por persona mientras los servicios suben \$90.000 es perder plata con cada inquilino de más , y hoy nada lo advierte. Por eso <code>base_calculo</code> distingue lo que sale de comparar con el mercado de lo que sale de la cuenta del propio inmueble.
MASCOTA, QUE SE ESTIMA DISTINTO	Personas y mascotas se cobran parecido y se estiman al revés. Una persona de más consume servicios; una mascota no consume casi nada y lo que trae es riesgo de desgaste : pisos, puertas, olores, la posibilidad de una reparación al final. Su precio no sale de una división sino de comparar con el mercado y, con el tiempo, de lo que Yalqui vea en las inspecciones de salida de unidades con mascota. Por eso <code>base_calculo = comparables</code> mientras no haya historia propia, y el número que importa acompañarlo es <code>mascotas_maximo</code> — un cargo por mascota sin tope convierte el apartamento en una perrera con recibo.
CÓMO SE USA	El propietario ve la sugerencia al lado de cada interruptor en la pantalla del canon y decide: la acepta, la sube o apaga la opción. Lo que quede se guarda en <code>inmueble_ajustes</code> , que es lo que de verdad se cobra. La valoración solo propone.
CUIDADO	Cobrar por persona es normal en habitaciones y apartaestudios, pero un cargo por cabeza sin relación con un costo real se parece mucho a cobrarle más a una familia por ser familia. <code>base_calculo = costo_marginal</code> con su <code>justificacion</code> deja el cargo explicado y defendible; un valor arbitrario, no.

M4 Demanda, verificación y aprobación

Desde que alguien pide una visita hasta que el propietario aprueba. Aquí es donde se enganchan Truora y las centrales de riesgo, sin que el resto del modelo sepa quién es el proveedor.

Al decidir, el propietario tiene **tres señales de naturaleza distinta**. Vale la pena no confundirlas, porque se usan de forma diferente y promediarlas sería un error.

SEÑAL	QUÉ DICE	CÓMO SE USA	TABLA
Ficha de caracterización	Quién dice ser y de qué vive	Contexto. Sirve para entender y emparejar. Nunca decide sola ni entra a ningún puntaje	<code>perfiles_inquilino</code>
Verificación externa	Identidad, antecedentes y comportamiento crediticio, según Truora y las centrales	Compuerta. Pasa o no pasa; con observaciones es una tercera opción que exige criterio humano	<code>verificaciones</code>
Score Yalqui	Cómo se comportó dentro de la plataforma	Gradiente. Solo existe si ya arrendó antes por Yalqui, y ausencia no es mal puntaje	<code>scores_inquilino</code>

030 Una compuerta y un gradiente no se promedian. Un inquilino con score 95 que no pasa la verificación de identidad no es «bueno con reservas»: no pasa. Y un inquilino sin score que pasa todas las verificaciones no está peor que uno con score 70 — está sin historia, que es distinto.

visitas FASE 1

Agendamiento de una visita al inmueble. El interesado puede no tener cuenta todavía.

CAMPOS	<code>id</code> , <code>inmueble_id</code> , <code>interesado_id</code> NULL, <code>nombre_contacto</code> , <code>telefono_contacto</code> , <code>email_contacto</code> , <code>inicio_at</code> , <code>fin_at</code> , <code>modalidad</code> enum <code>presencial·virtual</code> , <code>estado</code> enum <code>solicitada·confirmada·reprogramada·realizada·cancelada·no_asistio</code> , <code>reprogramada_de_id</code> self, <code>notas</code> , <code>created_at</code>
ÍNDICES	IDX(<code>inmueble_id</code> , <code>inicio_at</code>) · IDX(<code>estado</code> , <code>inicio_at</code>)

precalificaciones

FASE 1

El paso que faltaba entre la visita y la aplicación. Al terminar de mostrar la unidad, el propietario manda un enlace y en dos minutos sabe si a esa persona le alcanza.

CAMPOS	<code>id, visita_id, inmueble_id, interesado_id</code> NULL, token CHAR(43) UNIQUE, token_expira_at, nombre_completo, tipo_documento, numero_documento, fecha_nacimiento DATE, ciudad_nacimiento, genero enum, ocupacion enum estudiante·empleado·independiente·pensionado, ingresos_declarados DECIMAL(14,2), ingresos_aportantes DECIMAL(14,2), ingresos_totales DECIMAL(14,2), ingreso_verificado DECIMAL(14,2) NULL, metodo_ingreso enum declarado·extractos·nomina, antiguedad_laboral_meses, variabilidad_ingreso DECIMAL(5,2) NULL, cuota_creditos DECIMAL(14,2) NULL, canon_evaluado DECIMAL(14,2), gastos_unidad DECIMAL(14,2), disponible_estimado DECIMAL(14,2), num_dependientes, canon_anterior DECIMAL(14,2) NULL, motivo_mudanza, relacion_pct DECIMAL(5,2), <code>regla_id</code> , nivel enum holgado·ajustado·al_limite·no_alcanza, razones JSON, requiere_revision BOOL, estado enum enviada·en_diligenciamiento·preaprobada·con_reservas·no_alcanza·expirada, enviada_at, completada_at
ÍNDICES	UNIQUE(token) · IDX(inmueble_id, estado) · IDX(visita_id)
CÓMO FUNCIONA	El interesado abre el enlace y llena sus datos. El sistema calcula <code>relacion_pct</code> = <code>canon</code> ÷ <code>ingresos_totales</code> y la compara contra el tope. Si pasa, queda preaprobada y le llega otro enlace para subir documentos. Si no pasa, le pregunta quién va a ayudarle a pagar; a esa persona le llega su propio enlace para confirmar y declarar sus ingresos, y solo con esa confirmación se suman y se vuelve a evaluar.
OJO	Preaprobado no es aprobado. Los ingresos son <i>declarados</i> : nadie los ha probado todavía. Lo que los prueba son los extractos y el certificado laboral que vienen después, y ahí la cifra puede no coincidir. El estado tiene que leerse en la interfaz como «te alcanza, ahora demostralo», nunca como una promesa.
DATOS Y DECISIÓN	Edad, género y ciudad de nacimiento se piden porque los necesita la verificación de identidad y la ficha de caracterización — no entran en el cálculo . La preaprobación sale de una sola cuenta: canon contra ingresos. Que la decisión ahora sea automática lo hace más importante, no menos: un sesgo escrito en código se aplica a todos y sin que nadie lo note.

reglas_precalificacion

FASE 1

El criterio como datos y con versiones. Empieza siendo una sola condición y crece sin que nadie toque código.

CAMPOS	<code>id</code> , codigo, nombre, version SMALLINT, condiciones JSON, umbral_holgado DECIMAL(5,2), umbral_ajustado, umbral_limite, exige_aportante_desde DECIMAL(5,2), estado enum borrador·vigente·archivada, vigente_desde, vigente_hasta, <code>creada_por_id</code>
ÍNDICES	UNIQUE(codigo, version) · IDX(estado)
NOTAS	Cada precalificación guarda el <code>regla_id</code> con el que se evaluó. Sin eso no se puede explicar una decisión de hace ocho meses , que es exactamente lo que pide el habeas data cuando alguien reclama. Cambiar un umbral crea versión nueva y archiva la anterior; las precalificaciones viejas siguen apuntando a la suya.

analisis_ia

FASE 2

Cada corrida de un modelo sobre un documento o una solicitud, con su entrada, su salida y su confianza. Es lo que hace auditable todo lo que la IA toque.

CAMPOS	<code>id</code> , <code>entidad_tipo</code> enum <code>precalificacion_documento incidencian_foto inmueble_foto</code> , <code>entidad_id</code> , <code>tarea</code> enum <code>extraccion normalizacion_ingreso inconsistencia autenticidad_documento resumen riesgo</code> , <code>modelo</code> , <code>version_modelo</code> , <code>entrada_hash</code> CHAR(64), <code>salida</code> JSON, <code>confianza</code> DECIMAL(4,3), <code>hallazgos</code> JSON, <code>requiere_revision_humana</code> BOOL, <code>revisado_por_id</code> NULL, <code>revisado_at</code> , <code>decision_humana</code> enum <code>confirmada corregida descartada</code> , <code>costo</code> DECIMAL(10,4), <code>created_at</code>
ÍNDICES	<code>IDX(entidad_tipo, entidad_id)</code> · <code>IDX(tarea, requiere_revision_humana)</code> — la bandeja de lo que un humano debe mirar
NOTAS	<code>entrada_hash</code> permite no reprocesar el mismo documento dos veces y detectar si alguien lo cambió después. <code>decision_humana</code> es lo que con el tiempo se vuelve el conjunto de entrenamiento: cada corrección de una persona es una etiqueta. Guardar el <code>costo</code> importa porque estos modelos se pagan por uso y hay que saber cuánto cuesta procesar una aplicación.
REGLA	Ningún <code>analisis_ia</code> decide por sí solo. Alimenta la regla o le avisa a una persona. La <code>tarea riesgo</code> nace deshabilitada: no hay con qué entrenarla todavía.

precalificacion_aportantes

FASE 1

Quién más va a poner. Aparecen solo cuando al interesado no le alcanza solo.

CAMPOS	<code>id</code> , <code>precalificacion_id</code> , <code>nombre</code> , <code>relacion</code> enum <code>madre padre pareja hermano familiar empleador amigo otro</code> , <code>tipo_documento</code> , <code>numero_documento</code> , <code>telefono</code> , <code>ocupacion</code> enum, <code>ingresos_declarados</code> DECIMAL(14,2), <code>token</code> CHAR(43) UNIQUE, <code>token_expira_at</code> , <code>estado</code> enum <code>pendiente confirmado rechazado expirado</code> , <code>acepta_ser_codeudor</code> BOOL, <code>consentimiento_id</code> NULL, <code>enviado_at</code> , <code>confirmado_at</code> , <code>created_at</code>
ÍNDICES	<code>IDX(precalificacion_id)</code>
LO CONFIRMA ÉL, NO EL APLICANTE	El <code>aportante</code> recibe su propio enlace y responde por sí mismo. Que el <code>aplicante</code> marque «mi mamá acepta» es una afirmación de parte interesada sobre un tercero que no ha dicho nada — no sirve como compromiso ni alcanza para tratar sus datos. Con el enlace, la mamá declara <i>sus</i> ingresos, dice si acepta ser codeudora y autoriza el tratamiento. Solo entonces sus ingresos entran en la suma.
ESTADOS	<code>pendiente</code> mientras no responde — y ahí la precalificación queda en espera, no preaprobada. <code>rechazado</code> es información valiosa y no un error: significa que el <code>aplicante</code> contaba con alguien que no va a estar. El propietario lo sabe antes de aprobar y no cuando toque cobrar. Si <code>acepta_ser_codeudor</code> es verdadero, al firmar el contrato la fila se copia a <code>garantes</code> sin volver a pedir nada.

aplicaciones FASE 1

Postulación formal de un inquilino a un inmueble. Es el expediente que el propietario aprueba o rechaza.

CAMPOS	<code>id</code> , <code>inmueble_id</code> , <code>inquilino_id</code> , <code>precalificacion_id</code> NULL, <code>estado</code> enum <code>borrador·enviada·en_verificacion·en_negociacion·aprobada·rechazada·retirada·convertida</code> , <code>canon_ofrecido</code> DECIMAL(14,2), <code>fecha_ingreso_deseada</code> , <code>num_ocupantes</code> , <code>num_mascotas</code> TINYINT, <code>mensaje</code> TEXT, <code>enviada_at</code> , <code>decidida_at</code> , <code>decidida_por_id</code> , <code>motivo_rechazo</code> , <code>contrato_id</code> NULL, <code>created_at</code> , <code>updated_at</code>
ÍNDICES	IDX(<code>inmueble_id</code> , <code>estado</code>) · IDX(<code>inquilino_id</code> , <code>estado</code>)
NOTAS	<code>canon_ofrecido</code> es el resultado del <code>canon_base</code> más lo que el aplicante haya marcado en <code>aplicacion_ajustes</code> , y puede diferir de lo que el propietario esperaba: ahí nace Yalqui Negotiate, y también la negociación más común, que no es bajar el canon sino quitar el parqueadero. Al decidir, el propietario ve tres cosas distintas: la ficha de caracterización, el resultado de verificaciones (crédito y antecedentes) y el <code>scores_inquilino</code> vigente si el inquilino ya arrendó antes por Yalqui. Un inquilino nuevo no tiene score, y eso no es lo mismo que tener uno malo — el modelo lo distingue con NULL. Y «no autorizó compartirlo» es un tercer estado distinto de los dos anteriores: sale de consentimientos, no de <code>scores_inquilino</code> .
COMPARAR, NO SOLO EVALUAR	La pantalla de evaluación está resuelta para una aplicación. Falta la vista donde de verdad se decide: las tres que llegaron a la misma unidad, lado a lado — canon ofrecido, holgura de ingresos de la precalificación, resultado de verificación, score si lo tienen y qué ajustes pidió cada uno. No hace falta ninguna tabla nueva: es una consulta por <code>inmueble_id</code> con estado <code>IN</code> (<code>enviada</code> , <code>en_verificacion</code>). Y sin recomendar a nadie: solo poniéndolos en la misma pantalla, porque un propietario suele preferir un buen inquilino a cien mil pesos más y hoy el sistema los muestra igual de bien.

aplicacion_ajustes FASE 1

Qué servicios adicionales pide el aplicante. Es lo que convierte «ofrezco \$3.150.000» en una cifra comparable con la de otro aplicante.

CAMPOS	<code>id</code> , <code>aplicacion_id</code> , <code>ajuste_id</code> , <code>cantidad</code> SMALLINT, <code>valor_unitario</code> DECIMAL(14,2), <code>valor_total</code> DECIMAL(14,2)
ÍNDICES	UNIQUE(<code>aplicacion_id</code> , <code>ajuste_id</code>)
NOTAS	Faltaba. El precio se armaba dos veces con reglas distintas: el propietario configuraba las opciones en <code>inmueble_ajustes</code> y se congelaban en <code>contrato_ajustes</code> , pero entre esos dos momentos el inquilino elige — trae mascota, quiere el parqueadero, entran dos personas más — y eso no quedaba en ninguna parte. Sin esta tabla, dos aplicantes que ofrecen el mismo canon no son comparables, porque uno quiere parqueadero y el otro no.
CÓMO FLUYE	El aplicante marca opciones sobre <code>inmueble_ajustes</code> y el sistema calcula el total; al aprobar, estas filas se copian a <code>contrato_ajustes</code> con el precio congelado. Un solo camino de cálculo desde el aviso hasta la factura: publicar, aplicar, firmar y cobrar usan la misma suma.

aplicacion_documentos FASE 1

Los soportes que el inquilino sube y que alguien tiene que revisar: cédula, certificado laboral, extractos, declaración de renta.

CAMPOS	<code>id</code> , <code>aplicacion_id</code> , <code>tipo</code> enum <code>documento_identidad·certificado_laboral·extractos_bancarios·declaracion_renta·referencia·rut·otro</code> , <code>archivo_id</code> , <code>obligatorio</code> BOOL, <code>estado_revision</code> enum <code>pendiente·aceptado·rechazado</code> , <code>revisado_por_id</code> , <code>revisado_at</code> , <code>nota</code> , <code>created_at</code>
ÍNDICES	IDX(<code>aplicacion_id</code> , <code>estado_revision</code>) — la lista de lo que falta por revisar
NOTAS	Sube a fase 1. Estaba en fase 2 y era incoherente: si el inquilino puede aplicar desde la fase 1, tiene que poder subir los soportes desde la fase 1, o la aplicación llega vacía y el propietario no tiene con qué decidir. <code>obligatorio</code> permite que el propietario exija ciertos tipos según la unidad — un local comercial pide RUT, un apartamento no. <code>estado_revision</code> es revisión humana y no verificación externa: eso último es <code>verificaciones</code> , que sí entra en fase 2. En la fase 1 el propietario mira los papeles con sus propios ojos.

verificaciones FASE 2

Yalqui Screening. Una fila por consulta a una fuente externa sobre **una persona** — inquilino o garante — con su vigencia.

CAMPOS	<code>id</code> , <code>usuario_id</code> NULL, <code>garante_id</code> NULL, <code>tipo_documento</code> , <code>numero_documento</code> , <code>tipo</code> enum <code>identidad·documento·antecedentes_judiciales·antecedentes_policiales·listas_restrictivas·centrales_riesgo·laboral·proveedor_id</code> , <code>consentimiento_id</code> , <code>aplicacion_origen_id</code> NULL, <code>id_externo</code> , <code>estado</code> enum <code>solicitada·en_proceso·completada·fallida·expirada</code> , <code>resultado</code> enum <code>aprobado·con_observaciones·rechazado</code> , <code>score</code> DE <code>hallazgos</code> JSON, <code>respuesta</code> JSON, <code>costo</code> DECIMAL(10,2), <code>servicio_contratado_id</code> NULL, <code>solicitada_at</code> , <code>completada_at</code> , <code>webhook_recibido_at</code> , <code>vigente_hasta</code> DATE
ÍNDICES	UNIQUE(<code>proveedor_id</code> , <code>id_externo</code>) — el webhook llega más de una vez · IDX(<code>usuario_id</code> , <code>tipo</code> , <code>vigente_hasta</code>) — búsqueda de verificación reutilizable · IDX(<code>estado</code> , <code>solicitada_at</code>)
NOTAS	Cambio respecto de la revisión anterior: la verificación cuelga de la persona, no de la aplicación. Tenerla atada a <code>aplicacion_id</code> significaría que aplica a cinco inmuebles pagaba cinco veces la misma consulta de antecedentes — y Truora y las centrales cobran por cada consulta. Se consulta una vez, queda con <code>vigente_hasta</code> , y las aplicaciones la reutilizan mientras no expire. <code>aplicacion_origen_id</code> guarda cualquier cosa solo para saber quién la pagó.
TRUORA	Sus <i>checks</i> son asíncronos: se crea la consulta, devuelve un identificador y el resultado llega por webhook. Por eso <code>id_externo</code> , <code>estado</code> , <code>en_proceso</code> , y <code>webhook_recibido_at</code> . <code>hallazgos</code> guarda el detalle estructurado — qué lista, qué proceso, qué fecha — separado de <code>respuesta</code> es el volcado crudo. Lo primero se le muestra al propietario; lo segundo existe para auditar y para reprocesar si el proveedor cambia de
LOS DOS SCORES	El de aquí es externo y previo : lo que dicen Datacrédito o Truora antes de firmar. El de M10 es interno y posterior : lo que el inquilino hizo en Yalqui. Números distintos, escalas distintas, y no se promedian.

proveedores_verificacion FASE 2

Catálogo de las fuentes externas y qué sabe hacer cada una. Permite enrutar por tipo sin código: antecedentes a Truora, crédito a la central.

CAMPOS	<code>id</code> , <code>codigo</code> UNIQUE <code>truora·datacredito·transunion·experian·evidente</code> , <code>nombre</code> , <code>tipos_soportados</code> JSON, <code>modo</code> enum <code>api·webhook·manual</code> , <code>costo_consulta</code> DECIMAL(10,2), <code>vigencia_dias</code> SMALLINT, <code>requiere_consentimiento</code> BOOL, <code>activo</code> , <code>orden</code>
NOTAS	<code>vigencia_dias</code> por proveedor y por tipo es lo que alimenta verificaciones. <code>vigente_hasta</code> : una validación de identidad puede durar un año, una consulta a centrales de riesgo envejece en semanas. <code>costo_consulta</code> hace que el gasto de screening sea consultable y no una sorpresa a fin de mes. Nota práctica: Truora cubre identidad y antecedentes, pero no reporte crediticio — para eso hace falta la central, así que van a convivir dos proveedores y no uno.

aplicacion_verificaciones FASE 2

Qué verificaciones respaldaron esta aplicación. Es la consecuencia de haberlas hecho reutilizables.

CAMPOS	<code>id</code> , <code>aplicacion_id</code> , <code>verificacion_id</code> , <code>reutilizada</code> BOOL, <code>vigente_al_decidir</code> BOOL, <code>adjuntada_at</code>
ÍNDICES	UNIQUE(<code>aplicacion_id</code> , <code>verificacion_id</code>)
NOTAS	Sin esta tabla, con verificaciones reutilizables sería imposible reconstruir qué evidencia tenía el propietario cuando aprobó , que es exactamente lo que hay que poder mostrar si el arriendo termina mal. <code>vigente_al_decidir</code> se congela en el momento de la decisión: que hoy esté expirada no cambia que ese día no lo estaba.

garantes FASE 2

Codeudor, coarrendatario, fiador o póliza de arrendamiento. Indispensable en el mercado colombiano.

CAMPOS	<code>id</code> , <code>aplicacion_id</code> NULL, <code>contrato_id</code> NULL, tipo enum codeudor·coarrendatario·fiador·poliza, <code>usuario_id</code> NULL, nombre, tipo_documento, numero_documento, telefono, email, aseguradora, numero_poliza, vigencia_hasta
NOTAS	Nace atado a la aplicación y se hereda al contrato cuando esta se convierte.

negociaciones · negociacion_ofertas FASE 3

Yalqui Negotiate. Hilo de contraofertas sobre el canon inicial o sobre una renovación.

CAMPOS	negociaciones: <code>id</code> , <code>aplicacion_id</code> NULL, <code>contrato_id</code> NULL, tipo enum canon_inicial·renovacion, estado enum abierta·aceptada·rechazada·expirada, expira_at · ofertas: <code>id</code> , <code>negociacion_id</code> , <code>autor_id</code> , canon_propuesto, meses_propuestos, condiciones TEXT, estado, created_at
--------	--

M5 Contrato digital

El contrato es el otro eje del modelo: de él cuelga toda la facturación y buena parte de la operación. Las firmas se guardan con su evidencia, porque un contrato digital sin trazabilidad no sirve ante un juez.

plantillas_contrato FASE 1

El molde del que sale cada contrato. Faltaba: el modelo guardaba el PDF generado pero no de dónde había salido.

CAMPOS	<code>id</code> , codigo, nombre, marco_legal enum vivienda_urbana·comercial·habitacion·parqueadero·mixto, aplica_a_tipos JSON, cuerpo MEDIUMTEXT, variables JSON, version SMALLINT, estado enum borrador·vigente·archivada, <code>creada_por_id</code> , vigente_desde, vigente_hasta, created_at
ÍNDICES	UNIQUE(codigo, version) · IDX(marco_legal, estado)
NOTAS	marco_legal no es una etiqueta: en Colombia el arriendo de vivienda urbana se rige por la Ley 820 de 2003 y el de local comercial por el Código de Comercio, con reglas distintas de renovación, incremento y terminación. Una sola plantilla «de arrendamiento» para las dos cosas es un contrato mal hecho en uno de los dos casos. Por eso la plantilla se escoge desde <code>inmuebles.tipo</code> y no a mano.
VERSIONES	Editar una plantilla vigente no la modifica: crea una versión nueva y archiva la anterior. Los contratos ya firmados siguen apuntando a la versión con la que se firmaron, que es lo único que permite reconstruir años después qué decía exactamente el documento que alguien aceptó.

contratos FASE 1

Acuerdo vigente entre propietario e inquilino sobre un inmueble.

CAMPOS	<code>id</code> , <code>uuid</code> , <code>numero</code> , <code>inmueble_id</code> , <code>propietario_id</code> , <code>inquilino_id</code> , <code>aplicacion_id</code> NULL, <code>contrato_anterior_id</code> self, NULL, <code>plantilla_id</code> , <code>plantilla_version</code> SMALLINT, <code>clausulas_opcionales</code> JSON, <code>estado</code> enum <code>borrador</code> · <code>pendiente_firma</code> · <code>vigente</code> · <code>en_mora</code> · <code>en_terminacion</code> · <code>terminado</code> , <code>fecha_inicio</code> , <code>fecha_fin</code> , <code>meses_plazo</code> , <code>canon_mensual</code> DECIMAL(14,2), <code>valor_administracion</code> , <code>administracion_incluida</code> , <code>dia_pago</code> TINYINT, <code>garantia_tipo</code> enum <code>codeudor</code> · <code>poliza</code> · <code>fiador</code> · <code>deposito</code> · <code>ninguna</code> , <code>deposito</code> DECIMAL(14,2) NULL, <code>regimen_iva</code> enum <code>excluido</code> · <code>gravado</code> , <code>tarifa_iva</code> DECIMAL(5,2), <code>inquilino_agente_retenedor</code> BOOL, <code>tarifa_retencion</code> DECIMAL(5,2), <code>incremento_tipo</code> enum <code>ipc</code> · <code>ipc_mas_puntos</code> · <code>fijo</code> · <code>ninguno</code> , <code>incremento_valor</code> , <code>prorroga_automatica</code> , <code>archivo_id</code> , <code>hash_documento</code> CHAR(64), <code>firmado_at</code> , <code>terminado_at</code> , <code>motivo_terminacion</code> , <code>created_at</code> , <code>updated_at</code>
ÍNDICES	IDX(<code>estado</code> , <code>fecha_fin</code>) — alimenta las alertas de vencimiento · IDX(<code>inmueble_id</code>) · IDX(<code>inquilino_id</code>)
IMPUESTOS	Tres columnas que faltaban y que solo importan al final del año, cuando más duele no tenerlas. El arriendo de vivienda está excluido de IVA y el de local comercial está gravado: <code>regimen_iva</code> se deriva del <code>marco_legal</code> de la plantilla, así que el sistema ya sabe la respuesta y solo faltaba guardarla. Un propietario con un local abajo y apartamentos arriba tiene dos regímenes y hoy no se entera. <code>inquilino_agente_retenedor</code> cubre el caso del inquilino empresa que retiene sobre el canon: esa retención es impuesto que el propietario ya pagó y debe poder descontar, y sin la columna la factura dice un valor y el banco otro. Conviene confirmar tarifas y supuestos con un contador antes de construirlo.
GARANTÍA	<code>garantia_tipo</code> reemplaza al campo suelto de depósito. Hasta donde entiendo, la Ley 820 de 2003 prohíbe exigir depósitos en dinero en arrendamiento de vivienda urbana — la vía prevista es codeudor, fiador o póliza; en local comercial la situación es distinta. Si se confirma, la opción <code>deposito</code> debe quedar bloqueada cuando la plantilla es de vivienda, con una nota que explique por qué y ofrezca las alternativas. Ofrecer por defecto una cláusula que no debería ir es peor que no tener el campo: le da al propietario la impresión de que está bien.
NOTAS	<code>canon_mensual</code> es solo la base: lo que el inquilino paga sale de sumarle los <code>contrato_ajustes</code> vigentes del periodo. Guardarlo así y no como total permite subir el canon en la renovación sin recalcular cada ajuste, y permite quitar el parqueadero sin tocar el canon. <code>contrato_anterior_id</code> encadena renovaciones: cada renovación es un contrato nuevo, no una edición del anterior. Así el histórico de canon queda intacto.

contrato_ajustes FASE 1

Qué ajustes se pactaron en este contrato y a qué precio quedaron congelados. Es lo que la factura desglosa cada mes.

CAMPOS	<code>id</code> , <code>contrato_id</code> , <code>ajuste_id</code> , <code>cantidad</code> SMALLINT, <code>valor_unitario</code> DECIMAL(14,2), <code>valor_total</code> DECIMAL(14,2), <code>periodicidad</code> enum <code>mensual</code> · <code>unico</code> , <code>vigente_desde</code> DATE, <code>vigente_hasta</code> DATE NULL, <code>nota</code>
ÍNDICES	IDX(<code>contrato_id</code> , <code>vigente_desde</code>) · IDX(<code>ajuste_id</code>)
RELACIONES	N:1 → <code>contratos</code> · N:1 → <code>catalogo_ajustes</code> · 1:N → <code>factura_arriendo_conceptos</code>
NOTAS	<code>valor_unitario</code> se congela al firmar y no se lee de <code>inmueble_ajustes</code> : si el propietario sube el precio del parqueadero, los contratos vigentes no se mueven. <code>vigente_desde</code> y <code>vigente_hasta</code> permiten que un ajuste entre o salga a mitad de contrato — llega una mascota en marzo, se va el segundo ocupante en agosto — sin tocar el contrato ni renegociar el canon. El canon efectivo de un mes es <code>contratos.canon_mensual</code> más los ajustes mensuales vigentes en ese periodo.

incrementos_canon FASE 2

El aumento anual, con su cálculo y su constancia de notificación. Plata que hoy se pierde por olvido.

CAMPOS	<code>id</code> , <code>contrato_id</code> , <code>anio</code> SMALLINT, <code>aplica_desde</code> DATE, <code>indice</code> enum ipc·fijo·ninguno, <code>indice_valor</code> DECIMAL(5,2), <code>canon_anterior</code> DECIMAL(14,2), <code>canon_nuevo</code> DECIMAL(14,2), <code>estado</code> enum programado·notificado·aplicado·omitido·rechazado, <code>comunicado_id</code> NULL, <code>notificado_at</code> , <code>aplicado_at</code> , <code>omitido_motivo</code>
ÍNDICES	UNIQUE(<code>contrato_id</code> , <code>anio</code>) — un incremento por aniversario · IDX(<code>estado</code> , <code>aplica_desde</code>) — la cola de alertas
EL PROCESO, NO EL DATO	El dato ya estaba: <code>contratos</code> guarda <code>incremento_tipo</code> , <code>incremento_valor</code> y <code>fecha_inicio</code> , así que se sabe qué día toca subir cada canon y en cuánto. Lo que faltaba era el proceso. Sesenta días antes del aniversario se crea la fila en <code>programado</code> y sale la alerta con el IPC ya aplicado; el propietario aprueba, se genera el comunicado de notificación y la fila pasa a <code>notificado</code> ; al llegar la fecha, las facturas salen con el valor nuevo.
LA NOTIFICACIÓN IMPORTA	El incremento hay que avisarlo con anticipación y por un medio del que quede constancia. Por eso <code>comunicado_id</code> apunta al aviso enviado, que ya lleva confirmación de lectura. Sin esa constancia, un inquilino puede discutir el aumento y el propietario no tiene con qué.
TOPE LEGAL	En vivienda urbana el aumento está limitado al IPC del año anterior. <code>contratos.incremento_tipo</code> admite <code>ipc_mas_puntos</code> , que para vivienda no aplica: la opción debe quedar deshabilitada cuando la plantilla es de Ley 820 , igual que ya se elige la plantilla según el tipo de unidad. <code>omitido</code> existe para el propietario que decide no subir — es una decisión legítima que también conviene registrar.

contrato_firmas FASE 1

Una fila por firmante, con la evidencia que sostiene la validez de la firma electrónica.

CAMPOS	<code>id</code> , <code>contrato_id</code> , <code>rol_firma</code> enum propietario·socio_propietario·inquilino·garante·testigo, <code>usuario_id</code> NULL, <code>garante_id</code> NULL, <code>nombre</code> , <code>numero_documento</code> , <code>telefono</code> , <code>orden</code> TINYINT, <code>token_firma</code> CHAR(43) UNIQUE, <code>token_expira_at</code> , <code>enviado_at</code> , <code>visto_at</code> , <code>otp_enviado_at</code> , <code>otp_verificado</code> , <code>estado</code> enum pendiente·enviado·visto·firmado·rechazado·expirado, <code>firmado_at</code> , <code>ip</code> VARCHAR(45), <code>user_agent</code> , <code>proveedor_firma</code> , <code>evidencia</code> JSON
ÍNDICES	IDX(<code>contrato_id</code> , <code>estado</code>)
CÓMO SE FIRMA	Cada firmante recibe su propio link , no uno compartido: <code>token_firma</code> es único, de un solo uso y con vencimiento. Sale por WhatsApp usando una plantilla de <code>plantillas_mensaje</code> y el envío queda en <code>mensajes</code> . Al abrirlo se marca <code>visto_at</code> , se pide un OTP al mismo teléfono, y solo entonces se firma. La evidencia que hace defendible la firma — IP, dispositivo, hora, OTP verificado y el hash del documento exacto que se mostró — queda en la fila. <code>orden</code> permite firma secuencial cuando el negocio lo pida; por defecto todos pueden firmar en paralelo.
NOTAS	Los firmantes obligatorios se derivan de <code>inmueble_propietarios</code> : todo copropietario con <code>aparece_en_titulo</code> firma, o consta su poder en <code>contrato_anexos</code> . Un contrato firmado solo por uno de tres dueños registrados es exactamente el tipo de problema que aparece cuando ya hay que desalojar.

terminaciones FASE 2

La salida, con su lista de chequeo. El sistema acompaña la entrada en cinco etapas y soltaba la salida, que es donde se pierde plata y tiempo.

CAMPOS	<code>id</code> , <code>contrato_id</code> , <code>tipo</code> enum <code>vencimiento·mutuo·acuerdo·preaviso_inquilino·incumplimiento·restitucion</code> , <code>preaviso_recibido_at</code> , <code>fecha_salida_pactada</code> DATE, <code>fecha_salida_real</code> DATE NULL, <code>estado</code> enum <code>anunciada·en_curso·entregada·liquidada·cerrada</code> , <code>inspeccion_salida_id</code> NULL, <code>danos_atribuibles</code> DECIMAL(14,2), <code>saldo_pendiente</code> DECIMAL(14,2), <code>garantia_a_devolver</code> DECIMAL(14,2), <code>liquidacion_neta</code> DECIMAL(14,2), <code>checklist</code> JSON, <code>dias_vacia</code> SMALLINT NULL, <code>acta_archivo_id</code> NULL, <code>cerrada_at</code>
ÍNDICES	UNIQUE(<code>contrato_id</code>) · IDX(<code>estado</code> , <code>fecha_salida_pactada</code>)
LA LISTA	<code>checklist</code> lleva las ocho tareas encadenadas que hoy viven en la cabeza del propietario: inspección de salida contra la de entrada, daños atribuibles con su costo, paz y salvo de servicios y administración, entrega de llaves, cambio de titular de las cuentas de cobro, liquidación de lo que se devuelve, fotos nuevas y republicar. Ninguna es difícil; el problema es que son ocho, con plazos legales de por medio, y no había nada que las sostuviera.
EL DATO QUE NADIE CALCULA	<code>dias_vacia</code> — cuántos días quedó la unidad sin arrendar entre un contrato y el siguiente. Es la métrica que de verdad mide si el sistema sirve, y hoy no existe en ninguna parte. Se llena al firmarse el contrato siguiente.
NOTAS	Las inspecciones suben a fase 2 con esto: sin la de salida comparada contra la de entrada, los daños atribuibles son una opinión. Y la liquidación no la mueve Yalqui — como con el canon, calcula y deja constancia; la plata la devuelve el propietario.

contrato_anexos FASE 2

Documentos que se agregan durante la vida del contrato.

CAMPOS	<code>id</code> , <code>contrato_id</code> , <code>tipo</code> enum <code>inventario_entrega·otrosi·acta_entrega·paz_y_salvo·requerimiento</code> , <code>archivo_id</code> , <code>descripcion</code> , <code>created_at</code>
--------	--

M6 Flujo A · canon DIRECTO, FUERA DE YALQUI

Ninguna tabla de este módulo referencia una del módulo M7. El patrón sigue siendo el de cuentas por cobrar — la factura declara la deuda, el pago la abona, el saldo es la diferencia — pero con una vuelta de tuerca: el pago no lo ejecuta Yalqui, lo **reporta** alguien y hay que verificarlo antes de darlo por bueno.

facturas_arriendo FASE 1

Lo que el inquilino debe en un periodo. La genera la cobranza automática a partir del contrato.

CAMPOS	<code>id</code> , <code>contrato_id</code> , <code>periodo</code> CHAR(7) «2026-08», <code>fecha_emision</code> , <code>fecha_vencimiento</code> , <code>subtotal</code> DECIMAL(14,2), <code>mora</code> , <code>total</code> , <code>saldo</code> , <code>estado</code> enum <code>borrador·emitida·parcial·pagada·vencida·anulada</code> , <code>dias_mora</code> , <code>url_pago</code> , <code>proveedor_link_pago</code> , <code>referencia_link</code> , <code>link_expira_at</code> , <code>created_at</code> , <code>updated_at</code>
ÍNDICES	UNIQUE(<code>contrato_id</code> , <code>periodo</code>) — impide doble cobro del mismo mes · IDX(<code>estado</code> , <code>fecha_vencimiento</code>)
NOTAS	<code>saldo</code> es redundante frente a la suma de pagos, y a propósito: es la columna sobre la que corren las alertas de mora sin recalcular nada — y solo la mueven los pagos verificados . Las cuatro columnas de link son opcionales: si Yalqui genera link de pago para esa factura, la plata sigue cayendo en la cuenta del propietario, el link solo facilita la transferencia y deja referencia para conciliar.

factura_arriendo_conceptos FASE 1

El desglose de la factura. Sin esto no se puede explicar por qué un mes costó más.

CAMPOS	<code>id</code> , <code>factura_id</code> , concepto <code>enum</code> <code>canon·ajuste·administracion·servicios_publicos·mora·reparacion·descuento·otro</code> , <code>contrato_ajuste_id</code> <code>NULL</code> , descripción, valor <code>DECIMAL(14,2)</code>
NOTAS	Los descuentos van con valor negativo; no hay una tabla aparte para ellos. Los conceptos <code>ajuste</code> llevan <code>contrato_ajuste_id</code> , y por eso el inquilino puede tocar la línea «Parqueadero cubierto» en la factura y ver de dónde sale, en qué contrato se pactó y desde cuándo. Una factura sin desglose sería más fácil de generar y volvería inútil todo el modelo de canon configurable.

pagos_arriendo FASE 1

El comprobante de un pago que ocurrió por fuera de Yalqui, con su ciclo de verificación. No es una transacción: es evidencia con estado.

CAMPOS	<code>id</code> , <code>factura_id</code> , <code>contrato_id</code> , <code>reportado_por_id</code> , monto <code>DECIMAL(14,2)</code> , <code>fecha_pago_declarada</code> <code>DATE</code> , canal <code>enum</code> <code>link_pago·transferencia·consignacion·efectivo·otro</code> , <code>banco_origen</code> , <code>referencia_externa</code> <code>NULL</code> , <code>comprobante_archivo_id</code> , estado <code>enum</code> <code>reportado·en_verificacion·verificado·rechazado·reversado</code> , <code>verificado_como</code> <code>enum</code> <code>manual·regla·conciliacion·pasarela</code> , <code>verificado_por_id</code> <code>NULL</code> , <code>regla_id</code> <code>NULL</code> , <code>movimiento_bancario_id</code> <code>NULL</code> , <code>verificado_at</code> , <code>motivo_rechazo</code> , <code>retencion_fuente</code> <code>DECIMAL(14,2)</code> , <code>neto_recibido</code> <code>DECIMAL(14,2)</code> , <code>created_at</code> , <code>updated_at</code>
ÍNDICES	<code>UNIQUE(proveedor_link, referencia_externa)</code> donde no sea nula – idempotencia si el link de pago manda webhook · <code>IDX(factura_id, estado)</code> · <code>IDX(estado, created_at)</code> – la bandeja de pagos por verificar
NOTAS	Solo verificado mueve el saldo de la factura. Un pago por <code>link_pago</code> puede llegar a verificado automáticamente con el webhook de la pasarela; uno por transferencia o efectivo necesita que el propietario (o un admin) lo confirme, y ahí es donde <code>comprobante_archivo_id</code> deja de ser opcional. <code>rechazado</code> guarda el motivo y deja la factura como estaba, no la anula.

reglas_verificacion_pago FASE 2

«Si el monto coincide exacto y llegó entre el 1 y el 7, verifiqué solo.» Lo que convierte media hora mensual en un minuto.

CAMPOS	<code>id</code> , <code>propietario_id</code> , <code>inmueble_id</code> <code>NULL</code> , nombre, monto_exacto <code>BOOL</code> , tolerancia <code>DECIMAL(14,2)</code> , <code>dia_desde</code> <code>TINYINT</code> , <code>dia_hasta</code> <code>TINYINT</code> , canales <code>JSON</code> , <code>exige_comprobante</code> <code>BOOL</code> , <code>accion</code> <code>enum</code> <code>verificar·marcar_probable</code> , <code>activa</code> , <code>created_at</code>
ÍNDICES	<code>IDX(propietario_id, activa)</code> · <code>IDX(inmueble_id)</code>
NOTAS	La regla se guarda por propietario y no en el código porque cada uno tiene su tolerancia: hay quien acepta mil pesos de diferencia y quien quiere el peso exacto. <code>accion = marcar_probable</code> es el escalón para el desconfiado — no verifica, pero deja separados y en verde los pagos que cumplen, listos para aprobarse en lote con un clic.
CUIDADO	Una regla que verifica sola es la única parte del sistema donde el saldo se mueve sin que un humano mire. Por eso queda <code>pagos_arriendo.regla_id</code> : si algo salió mal se sabe cuál regla lo dejó pasar y se pueden revertir todos los que tocó.

movimientos_bancarios FASE 3

Lo que dice el banco. Cruzarlo contra lo que dicen los inquilinos es lo que **elimina** el trabajo de verificar, en vez de aliviarlo.

CAMPOS	id , propietario_id , banco, cuenta_enmascarada, fecha DATE, valor DECIMAL(14,2), tipo enum credito·debito, referencia, descripcion, origen enum extracto·api_bancaria·manual, archivo_origen_id NULL, estado enum sin_conciliar·conciliado·ignorado, pago_arriendo_id NULL, confianza DECIMAL(4,3), conciliado_at
ÍNDICES	UNIQUE(propietario_id, banco, fecha, valor, referencia) — impide cargar dos veces el mismo extracto · IDX(propietario_id, estado, fecha)
CÓMO CRUZA	Por monto, fecha y referencia contra los pagos reportados. confianza separa lo que se puede verificar solo de lo que hay que mirar: monto exacto con referencia coincidente es alta; monto exacto dentro de una ventana de días es media; monto parecido es apenas una sugerencia. Solo la alta verifica automáticamente.
LO QUE NADIE VE HOY	Empieza con el propietario subiendo el extracto y el mismo esquema sirve el día que haya API bancaria: solo cambia origen. Y trae un beneficio no obvio — los créditos sin conciliar son plata que entró y no corresponde a ningún pago reportado: un inquilino que pagó y no avisó, que hoy aparece como moroso.

M7 Flujo B · monetización de Yalqui PROPIETARIO → YALQUI

Planes y servicios a la carta. La estructura es simétrica a la del módulo anterior pero completamente independiente: otra contraparte, otro IVA, otra conciliación — y a diferencia del canon, este dinero sí lo cobra Yalqui. Todo se cobra por unidad: la suscripción y también los servicios a la carta. Un propietario con ocho unidades tiene ocho suscripciones, que pueden estar en planes distintos, y si contrata Legal para todas son ocho líneas en la factura.

planes FASE 1

Catálogo de planes. Es una tabla con seeders, no un enum, para que renombrar o repreciar no requiera migración.

CAMPOS	id , codigo UNIQUE basico·profesional·empresarial·corporativo, nombre, descripcion, precio_mes DECIMAL(14,2), moneda CHAR(3) = COP, ciclo_default enum mensual·anual, activo, orden, vigente_desde, vigente_hasta
NOTAS	Seeders con precios ya definidos, por unidad y por mes: Básico \$0, Profesional \$20.000, Empresarial \$25.000, Corporativo \$30.000 . Como la suscripción es por inmueble, precio_mes ya es el precio por unidad y no hacen falta min_unidades/max_unidades : si un plan solo aplica a partir de cierto portafolio, eso se valida en la aplicación, no en el catálogo. Los precios viejos no se editan: se cierran con vigente_hasta y se crea fila nueva, para no alterar lo ya facturado.

plan_caracteristicas FASE 2

Qué incluye cada plan y con qué tope. Es la tabla que habilita o bloquea funciones en la app.

CAMPOS	id , plan_id , caracteristica_codigo, incluida, limite NULL, nota
NOTAS	Sube a fase 2 por una consecuencia de las dos decisiones anteriores . Si el cliente es un propietario individual y la suscripción es por unidad, entonces los cuatro planes no se pueden diferenciar por tamaño de portafolio ni por número de usuarios: lo único que los separa es qué desbloquea cada uno en esa unidad — cuántas fotos, si trae cobranza automática, si incluye screening, si da reportes de rentabilidad. Sin esta tabla, «Empresarial» y «Corporativo» son solo dos precios distintos por lo mismo.

descuentos_volumen FASE 2

Escalones por cantidad de unidades. Sin esto, el cliente que más valor obtiene es el que más paga por unidad — y su respuesta racional es dejar la mitad del portafolio en Básico.

CAMPOS	<code>id</code> , <code>plan_id</code> NULL, desde_unidades SMALLINT, hasta_unidades SMALLINT NULL, descuento_pct DECIMAL(5,2), activo, vigente_desde, vigente_hasta
ÍNDICES	IDX(plan_id, desde_unidades)
NOTAS	No toca ninguna tabla existente: suscripciones ya guarda precio_congelado por unidad, así que el descuento se aplica al facturar contando las suscripciones activas del propietario. <code>plan_id</code> nulo significa que el escalón aplica a cualquier plan.
ES NEGOCIO, NO TÉCNICA	Ocho unidades en Corporativo son \$240.000 al mes. El propietario grande va a bajar a Básico las que menos producen, y con eso se rompe justo lo que hace bueno al sistema: tener el portafolio completo adentro para ver la rentabilidad real. El esquema aguanta la escala; falta decidirla.

suscripciones FASE 1

El plan de una unidad. Una suscripción por inmueble; el propietario aparece como dueño de todas las suyas.

CAMPOS	<code>id</code> , <code>inmueble_id</code> , <code>propietario_id</code> , <code>plan_id</code> , estado enum prueba·activa·morosa·cancelada·vencida, ciclo enum mensual·anual, precio_congelado DECIMAL(14,2), fecha_inicio, fecha_fin, proxima_facturacion_at, renovacion_automatica, cancelada_at, motivo_cancelacion, created_at, updated_at
ÍNDICES	UNIQUE(inmueble_id) donde estado IN (prueba, activa, morosa) — una sola suscripción viva por unidad · IDX(propietario_id) · IDX(estado, proxima_facturacion_at) — la cola de facturación
NOTAS	Se lleva propietario_id además de inmueble_id aunque sea derivable: es la columna sobre la que se agrupa la factura mensual, y evita un JOIN en la consulta más frecuente del módulo. Cada unidad puede estar en un plan distinto — típicamente varias en Básico y las que producen más, en Profesional. Cambiar de plan las ocho unidades a la vez es un UPDATE por propietario_id; se pierde la operación única que daba el modelo por propietario, y se gana granularidad real de cobro. precio_congelado protege al cliente de un cambio de tarifa a mitad de ciclo.

servicios FASE 2

Catálogo de servicios a la carta, con su modelo de cobro.

CAMPOS	<code>id</code> , codigo UNIQUE pricing_engine·legal·screening·negotiate·cobranza·seguro_arrendamiento·factoraje, nombre, descripcion, modelo_cobro enum unico·recurrente·por_uso·porcentaje, precio_base DECIMAL(14,2), porcentaje DECIMAL(6,3), moneda, requiere_contrato, activo
NOTAS	Factoraje y seguro de arrendamiento se cobran por porcentaje; screening y pricing, por uso. El modelo_cobro decide cómo se calcula el concepto de la factura — pero la base de cálculo es siempre una unidad, nunca el portafolio completo.

servicios_contratados FASE 2

La bisagra del modelo: qué servicio compró quién, sobre qué unidad, y a qué precio.

CAMPOS	<code>id</code> , <code>propietario_id</code> , <code>servicio_id</code> , <code>inmueble_id</code> NOT NULL, <code>contrato_id</code> NULL, estado enum solicitado·activo·completado·cancelado, precio_acordado DECIMAL(14,2), parametros JSON, solicitado_at, inicio_at, fin_at
RELACIONES	1:N ← valoraciones, verificaciones · 1:N ← factura_yalqui_conceptos
NOTAS	Un propietario en Básico puede contratar servicios: no hay validación contra el plan de la unidad, y una unidad gratuita puede generar factura de Yalqui por un Screening o un Pricing. Es la decisión que hace del plan gratuito una puerta de entrada al catálogo y no una demo. Todo servicio se cobra por unidad , así que inmueble_id es obligatorio: no existe el servicio de cuenta. Yalqui Legal contratado «para el propietario» son en realidad <i>n</i> filas, una por unidad, y así debe verse en la factura. contrato_id sigue opcional porque agrega precisión cuando el servicio se ata a un contrato puntual — Factoraje sobre las rentas de ese contrato, Legal sobre esa terminación — y no a la unidad en general.

facturas_yalqui FASE 2

Lo que el propietario le debe a Yalqui: un solo documento mensual que agrupa todas sus suscripciones y servicios. A diferencia del arriendo, esta sí es factura electrónica ante la DIAN.

CAMPOS	<code>id</code> , <code>propietario_id</code> , numero UNIQUE, periodo, fecha_emision, fecha_vencimiento, subtotal DECIMAL(14,2), impuestos, total, saldo, moneda, estado enum borrador·emitida·parcial·pagada·vencida·anulada, cufe NULL, <code>archivo_id</code> NULL, created_at
NOTAS	Una factura por propietario y periodo, con un concepto por cada unidad suscrita — no una factura por inmueble. Así el propietario de ocho unidades recibe un cobro, no ocho. cufe queda nulo hasta que exista la integración con DIAN; la columna se deja desde ya para no migrar después.

factura_yalqui_conceptos FASE 2

Desglose línea por línea: cada unidad suscrita es un concepto, cada servicio a la carta es otro.

CAMPOS	<code>id</code> , <code>factura_yalqui_id</code> , tipo enum suscripcion·servicio·ajuste, <code>suscripcion_id</code> NULL, <code>servicio_contratado_id</code> NULL, <code>inmueble_id</code> NULL, descripcion, cantidad, precio_unitario DECIMAL(14,2), tasa_impuesto DECIMAL(5,2), total
NOTAS	<code>inmueble_id</code> en el concepto es lo que permite prorratar el costo de Yalqui por unidad en movimientos sin adivinar. En la factura el propietario ve «Calle 93 #12-40 apto 302 — Profesional» y no una línea agregada.

pagos_yalqui FASE 2

Pagos del propietario a Yalqui. Mismo patrón que pagos_arriendo, tabla distinta.

CAMPOS	<code>id</code> , <code>factura_yalqui_id</code> , <code>propietario_id</code> , monto DECIMAL(14,2), metodo enum pse·tarjeta·debito_automatizado·transferencia, pasarela, referencia_externa, estado enum iniciado·aprobado·rechazado·reversado, pagado_at
ÍNDICES	UNIQUE(pasarela, referencia_externa)

M8 Operación e incidencias

incidencias FASE 2

Un daño o solicitud sobre un inmueble, desde el reporte hasta el cierre.

CAMPOS	<code>id</code> , <code>inmueble_id</code> NULL, <code>edificacion_id</code> NULL, <code>contrato_id</code> NULL, <code>reportada_por_id</code> , ambito enum unidad·area_comun, categoria enum plomeria·electrico·estructural·electrodomesticos·cerrajeria·humedad·otro, prioridad enum baja·media·alta·urgente, estado enum abierta·asignada·en_progreso·espera_aprobacion·resuelta·cerrada·rechazada, titulo, descripcion TEXT, responsable_costo enum propietario·inquilino·compartido·copropiedad·por_definir, <code>proveedor_id</code> NULL, costo_estimado DECIMAL(14,2), costo_final, reportada_at, sla_vence_at, resuelta_at, cerrada_at
ÍNDICES	IDX(estado, sla_vence_at) — alimenta las alertas de SLA · IDX(inmueble_id, reportada_at)
NOTAS	<code>responsable_costo</code> decide tres cosas a la vez: si el arreglo entra como concepto en la factura del inquilino, si entra como egreso del propietario en movimientos, y si genera un evento negativo en la dimensión cuidado_inmueble. Solo <code>responsable_costo = inquilino</code> afecta el score: un daño por desgaste normal no es culpa de nadie. Con el administrador del edificio en escena aparece la incidencia de <code>area_comun</code> — el ascensor, la bomba de agua, una filtración en el sótano. Esa no cuelga de ninguna unidad, así que <code>inmueble_id</code> pasa a ser nulo y manda <code>edificacion_id</code> . Una de las dos llaves siempre está.

incidencia_eventos FASE 2

Línea de tiempo de la incidencia: comentarios, cambios de estado, fotos, cierre.

CAMPOS `id, incidencia_id, autor_id, tipo enum comentario·cambio_estado·asignacion·cotizacion·foto·cierre, contenido TEXT, archivo_id NULL, created_at`

cotizaciones FASE 3

Presupuestos de proveedores sobre una incidencia, para que el propietario elija.

CAMPOS `id, incidencia_id, proveedor_id, monto DECIMAL(14,2), descripcion, validez_hasta, estado enum enviada·aceptada·rechazada·expirada, created_at`

M9 Riesgo, alertas y finanzas

alertas FASE 2

Todo lo que el sistema detecta y alguien tiene que atender: mora, contratos por vencer, SLA vencido, riesgo del inquilino.

CAMPOS `id, tipo enum mora·contrato_por_vencer·pago_fallido·incidencia_sla·riesgo_inquilino·score_actualizado·pago_por_verificar·suscripción·severidad enum info·media·alta·critica, entidad_tipo, entidad_id, destinatario_id, titulo, mensaje, estado enum generada_at, vista_at, atendida_at`

ÍNDICES `IDX(destinatario_id, estado, generada_at) · IDX(entidad_tipo, entidad_id)`

NOTAS La alerta riesgo_inquilino le avisa al propietario cuando scores_inquilino baja de nivel, no cuando el puntaje se mueve un decimal que va al inquilino en el mismo recálculo: nadie debería enterarse de que su puntaje bajó porque le negaron un arriendo. pago_por_verificar tiene un comprobante esperando, que es lo que traba la conciliación. Referencia genérica por entidad_tipo + entidad_id en vez de entidad_id del modelo donde se acepta ese compromiso, porque las alertas son transversales y desechables.

obligaciones_propietario FASE 2

La otra mitad del calendario. Hoy muestra lo que le deben al propietario y no lo que él debe.

CAMPOS `id, propietario_id, inmueble_id NULL, edificacion_id NULL, tipo enum impuesto_predial·poliza_arrendamiento·seguro_inmueble·revision_gas·certificado_gasodomicos·cuota_extraordinaria·descripcion, monto_estimado DECIMAL(14,2), fecha_vencimiento DATE, fecha_descuento DATE NULL, periodicidad enum unica·anual·quinquenal, recordar_dias_antes SMALLINT, estado enum pendiente·recordada·cumplida·vencida·omitida, movimiento_id NULL, soporte_archivo_id NULL, cumplida_at`

ÍNDICES `IDX(propietario_id, estado, fecha_vencimiento) — alimenta el calendario · IDX(inmueble_id)`

NOTAS fecha_descuento no es un adorno: el predial tiene descuento por pronto pago y perderlo es plata contante. periodicidad = quinquenal cubre la revisión de gas, que se olvida justamente porque pasa cada cinco años. Al cumplirse, movimiento_id la enlaza con el egreso de la unidad, entra sola a la rentabilidad de la unidad o de la edificación según de quién sea.

CASI NO CUESTA Se pinta en el calendario de pagos con los mismos tres colores que ya usa — vencido, cumplido, aún no llega — y del otro lado de la moneda. No hace falta una pantalla nueva.

expedientes_mora FASE 3

El paquete que pide un abogado, armado de lo que el sistema ya guarda. De lo poco del catálogo que un propietario pagaría sin dudarlo.

CAMPOS	<code>id</code> , <code>contrato_id</code> , <code>generado_por_id</code> , periodo_desde DATE, periodo_hasta DATE, meses_en_mora, total_adeudado DECIMAL(14,2), num_requerimientos, incluye JSON, <code>archivo_id</code> , hash_documento CHAR(64), generado_at
ÍNDICES	IDX(contrato_id, generado_at)
QUÉ LLEVA	Contrato firmado con su evidencia de firma · estado de cuenta mes a mes · cada requerimiento enviado con su fecha y su estado de entrega, sacado de mensajes · el histórico de comunicación · los pagos que sí llegaron. No hay nada que calcular: es una consulta sobre lo que ya está guardado, empaquetada en un PDF.
POR QUÉ VALE	Un propietario que va donde el abogado con capturas de pantalla de WhatsApp tiene un caso más débil que uno que llega con constancias de entrega fechadas. Yalqui es el único que tiene esa evidencia y hoy la deja repartida en cinco tablas sin forma de sacarla. Es lo que le da contenido a «Yalqui Legal», que hoy está en el catálogo de servicios sin nada detrás. hash_documento permite probar después que el expediente no se alteró.

movimientos FASE 3

Libro de ingresos y egresos, **en dos niveles:** la unidad y la edificación. Responde «¿cuánto me deja este apartaestudio?» sin obligar a inventar dónde va el costo de pintar la fachada.

CAMPOS	<code>id</code> , ambito enum unidad·edificacion, <code>inmueble_id</code> NULL, <code>edificacion_id</code> NULL, <code>contrato_id</code> NULL, <code>movimiento_padre_id</code> NULL, tipo enum ingreso·egreso, categoria enum canon·administracion·mantenimiento·fachada·zonas_comunes·impuesto_predial·seguro·suscripcion_yalqui·servicio_yalqui, monto DECIMAL(14,2), fecha, prorratio enum ninguno·partes_iguales·por_area·por_canon, origen_tipo enum pago_arriendo·incidencia·factura_yalqui·manual, origen_id, nota, created_at
ÍNDICES	IDX(inmueble_id, fecha) · IDX(edificacion_id, fecha) · IDX(movimiento_padre_id) · IDX(origen_tipo, origen_id)
EL GASTO DE LA EDIFICACIÓN	Pintar la fachada cuesta \$8.000.000 y no es de ningún apartaestudio. Se registra como una fila con ambito = edificacion y edificio_id = inmueble_id. Si el propietario quiere ver rentabilidad por unidad, elige un prorratio y el sistema inserta filas hijas con ambito = unidad apuntando por movimiento_padre_id. El reporte del edificio suma padres; el de cada unidad suma hijas más sus movimientos propios. Nada se crea y la fachada nunca deja de existir como gasto real de la edificación.
CÓMO SE REPARTE	partes_iguales divide entre unidades activas; por_area pondera por metros —lo más justo cuando los apartaestudios son distintos· por_canon carga más a las que más producen; ninguno deja el gasto solo a nivel edificación, que es válido si el dueño quiere mirar el como un bloque. Las hijas se materializan y no se recalculan: si el año entrante agrega dos unidades, el reparto de la pintada de este mueve.
NOTAS	Reconstruible desde las tablas transaccionales; si se corrompe, se regenera. Los ingresos entran solo desde pagos verificados . Perm registrar egresos que no pasan por Yalqui —predial, seguro, la pintada— que el propietario carga a mano. Las proyecciones se calculan en esta tabla y no se almacenan. Una fila válida tiene inmueble_id o edificacion_id, nunca ninguno de los dos.

M10

Perfil y scoring del inquilino

Dos cosas que conviene no confundir. La **caracterización** describe quién es el inquilino y sirve para entenderlo y emparejarlo con el inmueble adecuado. El **scoring** mide cómo se comportó, y de él dependen decisiones que le afectan la vida: si lo aprueban, si le renuevan, en qué condiciones.

Por eso el modelo los separa físicamente y no deja que se toquen:

perfiles_inquilino guarda lo primero, el subsistema de score guarda lo segundo, y ninguna dimensión del score puede alimentarse de un campo demográfico.

Las seis reglas del subsistema

Un score portable es la pieza con más poder sobre la vida del inquilino de todo el producto: ya no le cuesta un contrato, le cuesta todos los futuros. Estas reglas están en el esquema, no en la aplicación, porque una regla que vive solo en el código se rompe en el primer sprint apurado.

01 Ningún dato demográfico entra al puntaje

Edad, género, profesión, nivel educativo y hobbies describen y emparejan; nunca puntúan. Son categorías protegidas o proxies de ellas, y usarlas para negar un arriendo es discriminación además de un problema bajo la Ley 1581 de 2012. El esquema lo hace cumplir: no existe ninguna llave entre `perfiles_inquilino` y `dimensiones_score`, y `eventos_score` solo acepta eventos con un `origen_tipo` de comportamiento.

02 Nada entra al score sin evidencia rastreable

Todo evento apunta a la fila que lo originó con `origen_tipo` + `origen_id`. Los eventos manual son la excepción y por eso exigen `registrado_por_id` y quedan en auditoría: son los únicos que un humano puede inventar.

03 Nada negativo cuenta sin atribución ni descargos

Una PQRS solo pesa si llega a `resuelta_en_contra` después de que el señalado pudo responder. Una incidencia solo pesa si `responsable_costo = inquilino`. Una inspección de salida solo pesa contra la de entrada. El desgaste normal y la acusación sin resolver no son faltas.

04 Todo caduca

`dimensiones_score.ventana_meses` define cuánto tiempo cuenta un evento. Sin caducidad, un mal semestre persigue a alguien para siempre — y con el score portable, lo persigue en todas partes. Sigue el principio de caducidad del dato negativo de la Ley 1266 de 2008.

05 Todo evento es reclamable, y el porqué se dice en español

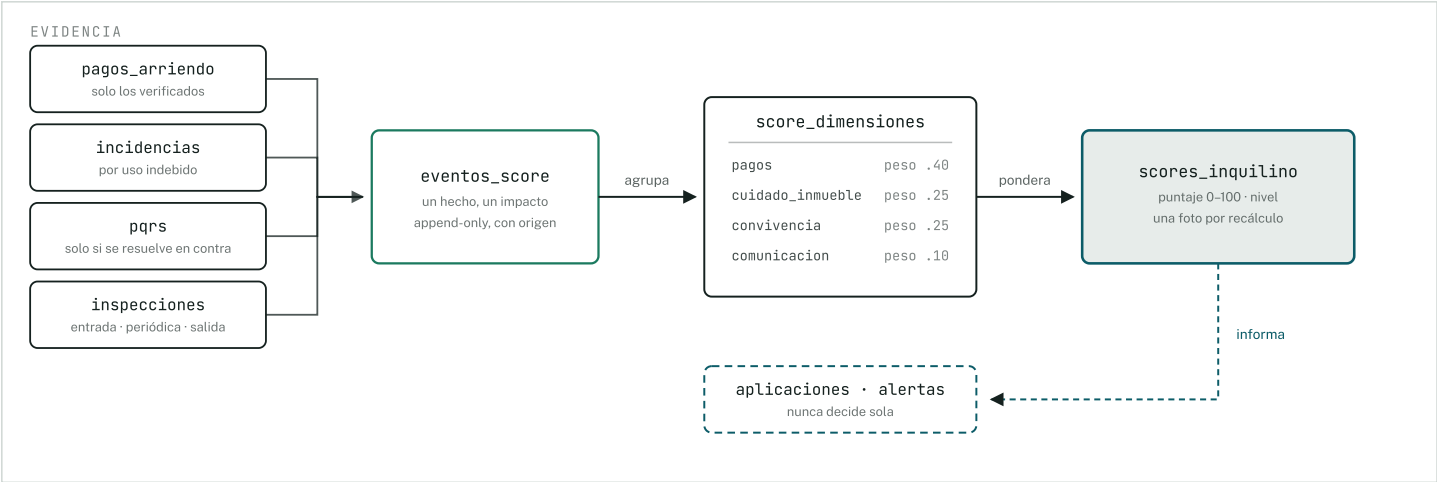
estado lleva el evento por `en_reclamo` hasta anulado o de vuelta a vigente; anulado nunca se borra. Y cada evento guarda una `explicacion_publica` en lenguaje llano: el inquilino lee «se registró el pago de agosto seis días después del vencimiento», `no_pago_tardio` · -2.5.

06 Ausencia de score no es score bajo

Un inquilino sin historia no tiene fila en `scores_inquilino`. Y «no autorizó compartirlo» es un tercer estado distinto, que sale de consentimientos. La interfaz tiene que distinguir los tres o el modelo castiga al que llega nuevo.

Cómo se construye el puntaje

El score no se escribe: se deriva. Cada hecho verificable deja un evento con su impacto; los eventos se agrupan por dimensión; las dimensiones se ponderan y dan un puntaje global. Igual que movimientos, todo es reconstruible desde la evidencia.



El puntaje es una consecuencia, no una entrada. Nadie escribe un score a mano: se recalcula desde eventos_score, y cada evento apunta a la fila que lo originó — un pago, una incidencia, una PQRS resuelta, un acta de inspección. Eso permite mostrarle al inquilino exactamente por qué bajó, que es lo que exige el habeas data, y permite recalcular todo el histórico si cambian los pesos.

Quién ve qué

El puntaje es portable; la evidencia no. Los eventos contienen direcciones, nombres y descripciones de hechos ocurridos en el inmueble de otro propietario — compartirlos sería filtrar los datos de un cliente a otro. Lo que viaja es el agregado.

QUIÉN	PUNTAJE Y NIVEL	DESGLOSE POR DIMENSIÓN	EVENTOS
El inquilino, sobre sí mismo	Sí	Sí	Sí, todos, con su origen y su fecha — y puede reclamar cualquiera
Propietario con contrato vigente	Sí	Sí	Solo los originados en su propio contrato
Propietario evaluando una aplicación	Sí, si hay compartir_score vigente	Sí, agregado	No
Admin de Yalqui	Sí	Sí	Sí, y el acceso queda en auditoria
Vecino que reporta una PQRS	No	No	No

NOTA Es una regla de acceso, no una columna: el esquema la habilita porque scores_inquilino y score_dimensiones son agregados sin datos identificables de terceros, mientras eventos_score siempre apunta a un contrato_id — que es exactamente lo que permite filtrarlos por propietario.

Tablas

inquilino_atributos

FASE 3

Hobbies, habilidades e idiomas del inquilino, normalizados para poder filtrar y emparejar.

CAMPOS	id, usuario_id, tipo enum hobby·habilidad·idioma·certificacion, valor VARCHAR(191), verificado, created_at
ÍNDICES	UNIQUE(usuario_id, tipo, valor) · IDX(tipo, valor) — para buscar por atributo
NOTAS	Hasta la fase 3 esto cabe como columna JSON en perfiles_inquilino; se normaliza cuando exista una pantalla que filtre por ello. Un propietario que arrienda un local puede querer un inquilino con cierta habilidad, y un edificio con zona de coworking puede querer emparejar por profesión — pero eso es emparejamiento , no calificación : nada de esto toca el score.

consentimientos FASE 1

Autorizaciones que el usuario otorga y puede revocar. Deja de ser opcional en el momento en que el score se vuelve portable.

CAMPOS	<code>id, usuario_id, tipo</code> enum <code>tratamiento_datos·compartir_score·consulta_centrales·consulta_antecedentes·comunicaciones, otorgado</code> BOOL, <code>version_politica, alcance</code> JSON, <code>otorgado_at, revocado_at, ip, user_agent, created_at</code>
ÍNDICES	<code>IDX(usuario_id, tipo, otorgado_at)</code> – se lee siempre el más reciente por tipo
NOTAS	Append-only: revocar no borra, inserta. Hacen falta tres autorizaciones distintas y no una sola casilla — tratar los datos, compartir el score con propietarios que evalúan, y consultar centrales de riesgo, que en Colombia exige autorización expresa y previa. <code>compartir_score</code> es la que habilita la portabilidad; sin ella vigente, un propietario que evalúa no ve nada, ni siquiera que el inquilino tiene historia.

dimensiones_score FASE 2

Catálogo de las dimensiones que se miden y cuánto pesa cada una. Es una tabla, no un enum, porque los pesos se van a calibrar con datos reales.

CAMPOS	<code>id, codigo</code> UNIQUE <code>pagos·cuidado_inmueble·convivencia·comunicacion·documentacion, nombre, descripcion, explicacion_publica</code> TEXT, <code>como_mejorar</code> TEXT, <code>peso</code> DECIMAL(4,3), <code>puntaje_base</code> DECIMAL(5,2), <code>ventana_meses</code> SMALLINT, <code>calculo</code> enum <code>automatico·manual·mixto, activo, orden, vigente_desde, vigente_hasta</code>
NOTAS	Las cuatro que nombraste más <code>documentacion</code> (si entrega a tiempo lo que se le pide). Los pesos deben sumar 1 entre las dimensiones vigentes — se valida en la aplicación, MySQL no puede. Cambiar un peso no reescribe el pasado: se cierra la fila con <code>vigente_hasta</code> y se crea otra, para que un score viejo se pueda explicar con los pesos que regían ese día. <code>ventana_meses</code> es la caducidad del dato negativo: los eventos más viejos que esa ventana dejan de contar. No es un adorno — es lo que evita que un mal semestre persiga a alguien para siempre ahora que el score es portable, y va en la misma línea del principio de caducidad de la Ley 1266 de 2008 para el dato financiero negativo.

eventos_score FASE 2

La tabla que de verdad importa: un hecho verificable, la dimensión que afecta y cuánto. Solo inserción, nunca actualización.

CAMPOS	<code>id, inquilino_id, contrato_id</code> NULL, <code>dimension_id, tipo_evento</code> enum <code>pago_puntual·pago_tardio·pago_incumplido·acuerdo_cumplido·dano_atribuible·inspeccion_favorable·inspeccion_desfav</code> <code>impacto</code> DECIMAL(5,2) con signo, <code>origen_tipo</code> enum <code>pago_arriendo·incidencia·pqrs·inspeccion·contrato·manual, origen</code> <code>motivo_anulacion, anulado_por_id</code> NULL, <code>anulado_at, ocurrido_at, created_at</code>
ÍNDICES	<code>IDX(inquilino_id, ocurrido_at)</code> · <code>IDX(origen_tipo, origen_id)</code> – evita duplicar el evento del mismo hecho · <code>IDX(dimension_id, ocurrido_at)</code>
NOTAS	<code>descripcion</code> es la nota interna y <code>explicacion_publica</code> es lo que lee el inquilino: se separan a propósito, porque redactar para una persona es exactamente una fila, por más veces que corra el recálculo. Los eventos <code>manual</code> exigen <code>registrado_por_id</code> y quedan en auditoria, pero el score portable deja de ser un tecnicismo: un evento injusto ya no le cuesta un contrato, le cuesta todos los futuros. Un reclamo pasado queda la constancia de por qué.

scores_inquilino FASE 2

Una foto del puntaje en un momento dado. Histórico, no mutable: cada recálculo inserta una fila nueva.

CAMPOS	<code>id</code> , <code>inquilino_id</code> , <code>puntaje_global</code> DECIMAL(5,2) 0-100, <code>puntaje_anterior</code> DECIMAL(5,2) NULL, <code>delta</code> DECIMAL(5,2), <code>nivel</code> enum excelente·bueno·regular·riesgo·critico, <code>num_contratos</code> , <code>meses_historia</code> , <code>num_eventos</code> , <code>version_modelo</code> , <code>vigente</code> BOOL, <code>calculado_at</code>
ÍNDICES	UNIQUE(<code>inquilino_id</code>) donde <code>vigente</code> = 1 — una sola foto vigente · IDX(<code>inquilino_id</code> , <code>calculado_at</code>)
NOTAS	Se guarda <code>meses_historia</code> y <code>num_eventos</code> junto al puntaje porque un 95 con dos meses de historia no significa lo mismo que un 95 con cuatro años, y el propietario tiene que poder verlo. Un inquilino sin historia no tiene fila: ausencia de score no es score bajo , y la interfaz tiene que distinguirlo. <code>puntaje_anterior</code> y <code>delta</code> se guardan calculados para que la app pueda decir «subiste 3 puntos este mes» sin ir a buscar la foto anterior — es la clase de detalle que separa un dashboard de algo que la gente entiende.

score_dimensiones FASE 2

El desglose de cada foto. Es lo que permite decir «paga impecable pero tiene tres quejas de convivencia» en vez de un número opaco.

CAMPOS	<code>id</code> , <code>score_id</code> , <code>dimension_id</code> , <code>puntaje</code> DECIMAL(5,2), <code>peso_aplicado</code> DECIMAL(4,3), <code>num_eventos</code> , <code>tendencia</code> enum sube·estable·baja
ÍNDICES	UNIQUE(<code>score_id</code> , <code>dimension_id</code>)
NOTAS	Se guarda <code>peso_aplicado</code> y no se lee del catálogo: así una foto vieja se sigue explicando con los pesos que tenía entonces, aunque hoy sean otros.

pqrs FASE 3

Peticiones, quejas, reclamos y sugerencias — típicamente de vecinos o de la administración del edificio, sobre la conducta en la unidad.

CAMPOS	<code>id</code> , <code>inmueble_id</code> , <code>contrato_id</code> NULL, <code>senalado_id</code> NULL, <code>tipo</code> enum peticion·queja·reclamo·sugerencia·felicitacion, <code>categoria</code> enum ruido·mascotas·basuras·parqueadero·areas_comunes·dano·convivencia·otro, <code>origen</code> enum vecino·administracion·propietario·inquilino·porteria, <code>reportante_nombre</code> , <code>reportante_anonimo</code> BOOL, <code>descripcion</code> TEXT, <code>gravedad</code> enum leve·moderada·grave, <code>estado</code> enum recibida·en_revision·en_descargos·resuelta_en_contra·resuelta_a_favor·desestimada, <code>descargos</code> TEXT, <code>resolucion</code> TEXT, <code>resuelta_por_id</code> , <code>confirmada_por_propietario</code> BOOL, <code>confirmada_at</code> , <code>recibida_at</code> , <code>resuelta_at</code>
ÍNDICES	IDX(<code>inmueble_id</code> , <code>recibida_at</code>) · IDX(<code>senalado_id</code> , <code>estado</code>)
NOTAS	Solo <code>resuelta_en_contra</code> y <code>confirmada_por_propietario</code> generan evento de score. El administrador del edificio resuelve para efectos de la convivencia; el dueño de la unidad confirma para efectos del score. Una queja anónima recibida no es una falta probada, y castigar al inquilino por acusaciones sin resolver convierte el score en un arma. Por eso existe el estado <code>en_descargos</code> : el señalado tiene que poder responder antes de que su puntaje se mueva. <code>felicitacion</code> existe a propósito — el score debería poder subir por buena convivencia, no solo bajar.

inspecciones FASE 2

Actas de estado del inmueble. Sin esto, «cuidado de la unidad» sería una opinión del propietario.

CAMPOS	<code>id</code> , <code>inmueble_id</code> , <code>contrato_id</code> , <code>tipo</code> enum entrada·periodica·salida, <code>realizada_por_id</code> , <code>fecha</code> , <code>estado_general</code> enum excelente·bueno·regular·malo, <code>puntaje</code> DECIMAL(5,2), <code>items</code> JSON, <code>observaciones</code> TEXT, <code>acta_archivo_id</code> , <code>firmada_por_inquilino</code> BOOL, <code>created_at</code>
NOTAS	La de entrada es la línea base; la de salida se compara contra ella y esa diferencia — no el estado absoluto — es la que genera el evento de score. Un apartamento entregado regular y devuelto regular no debería castigar a nadie. <code>firmada_por_inquilino</code> es lo que hace el acta oponible.

Comunicación y vecindad

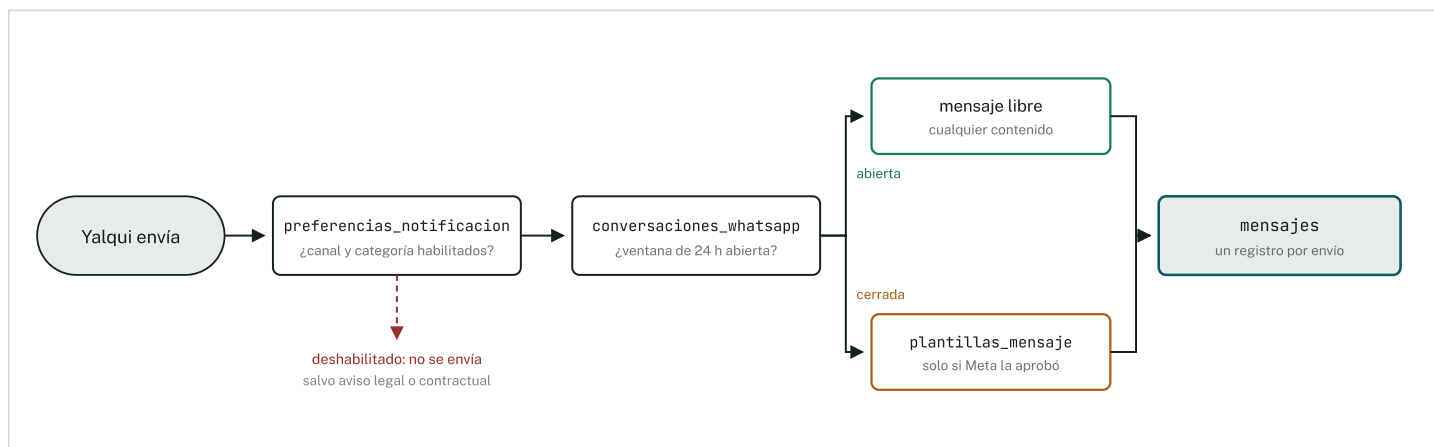
Hasta esta revisión el modelo mandaba mensajes desde tres lugares distintos y ninguno se hablaba con los otros: la cobranza tenía su propia bitácora, las incidencias su línea de tiempo, las alertas su bandeja. Un comunicado general no cabía en ninguno, y WhatsApp aparecía como un valor de enum en una tabla de cobranza — que es exactamente donde no debe estar.

Este módulo lo consolida: **un solo registro de salida** para todo lo que Yalqui le manda a alguien, por el canal que sea, con el motivo que sea. `gestiones_cobranza` desaparece y se convierte en filas de mensajes con `origen_tipo = cobranza`.

Por qué WhatsApp no es «un canal más»

Enviar por correo es mandar y ya. WhatsApp Business no funciona así, y si el esquema lo trata como si funcionara, el número se termina bloqueando. Meta separa dos modos según una ventana de 24 horas contada desde el último mensaje *entrante* de esa persona: dentro de la ventana se puede escribir libre; fuera de ella solo se pueden enviar **plantillas pre-aprobadas**, con una categoría declarada que determina el costo y el nivel de escrutinio.

Eso implica dos tablas que no existirían si solo mandáramos correos: una para el estado de la plantilla en Meta, otra para el estado de la ventana por teléfono.



El orden de las compuertas importa. Primero la preferencia del usuario, después la regla de Meta — nunca al revés, porque un mensaje técnicamente permitido que el usuario pidió no recibir sigue siendo spam. La única excepción son los avisos con obligación legal o contractual (mora, terminación, incremento del canon), que no se pueden desactivar: por eso `preferencias_notificacion` distingue categorías y no solo canales.

Tablas

mensajes FASE 1

Registro único de todo lo que sale de Yalqui hacia una persona, por el canal que sea. Absorbe la vieja gestiones_cobranza.

CAMPOS	<code>id</code> , canal enum whatsapp·email·sms·push·app, direccion enum saliente·entrante, <code>contrato_id</code> NULL, <code>responde_a_id</code> NULL, <code>destinatario_id</code> NULL, destinatario_telefono, destinatario_email, <code>plantilla_id</code> NULL, asunto, contenido_renderizado TEXT, origen_tipo enum comunicado·cobranza·incidencia·contrato·score·alerta·verificacion·ayuda_vecino·manual, origen_id, contexto JSON, proveedor, mensaje_id_externo, estado enum encolado·enviado·entregado·leido·fallido·rechazado, error, costo DECIMAL(10,4), enviado_at, entregado_at, leido_at, created_at
ÍNDICES	UNIQUE(proveedor, mensaje_id_externo) – los webhooks de estado llegan varias veces · IDX(contrato_id, created_at) – la bandeja por unidad · IDX(origen_tipo, origen_id) · IDX(destinatario_id, created_at) · IDX(canal, estado)
TAMBIÉN ENTRA	El sistema mandaba mensajes y no recibía ninguno: todo lo que el inquilino respondía se perdía por fuera, y con eso el contexto de lo que se acordó. Con direccion = entrante y contrato_id, cada hilo se agrupa por unidad y no por número de teléfono , que es como el propietario piensa. Y hay un detalle que juega a favor: cuando el inquilino escribe se abre la ventana de 24 horas y durante ese rato se le puede responder libre y sin costo de plantilla — hoy esa ventana se abre y se desperdicia porque nadie la mira.
NOTAS	contexto guarda lo que antes eran columnas sueltas de cobranza — dias_mora_al_enviar, el periodo, el saldo al momento del envío — sin obligar a una tabla por cada motivo de mensaje. costo importa: WhatsApp cobra por conversación iniciada por el negocio, y sin esta columna nadie sabe cuánto cuesta cobrarle a un moroso. El contenido se guarda renderizado , no como referencia a la plantilla: si la plantilla cambia el año que viene, el mensaje que se envió sigue diciendo lo que decía.

plantillas_mensaje FASE 2

Catálogo de plantillas por canal, con su estado de aprobación en Meta cuando el canal es WhatsApp.

CAMPOS	<code>id</code> , codigo UNIQUE, nombre, canal enum whatsapp·email·sms·push·app, categoria enum cobranza·comunicado·transaccional·incidencia·contrato·comercial, asunto, cuerpo TEXT, variables JSON, idioma CHAR(5), nombre_meta, categoria_meta enum utility·authentication·marketing, estado_aprobacion enum borrador·en_revision·aprobada·rechazada·pausada, motivo_rechazo, version, activo, created_at, updated_at
ÍNDICES	UNIQUE(codigo, idioma, version) · IDX(canal, estado_aprobacion)
NOTAS	categoria_meta no es lo mismo que categoria y hay que declararla bien: un aviso de corte de agua es utility, una promoción de plan es marketing. Etiquetar marketing como utility para ahorrar es la vía rápida a que Meta baje la calificación del número y termine limitando el envío. version permite editar una plantilla sin invalidar la aprobación de la anterior mientras la nueva se revisa.

conversaciones_whatsapp FASE 2

Estado de la ventana de 24 horas por número. Es lo que decide si se puede escribir libre o solo con plantilla.

CAMPOS	<code>id</code> , telefono UNIQUE, <code>usuario_id</code> NULL, ultimo_entrante_at, ventana_expira_at, estado enum abierta·cerrada·opt_out·bloqueado, opt_out_at, created_at, updated_at
ÍNDICES	UNIQUE(telefono) · IDX(ventana_expira_at)
NOTAS	El teléfono es la llave y no el usuario, porque un número puede escribir antes de tener cuenta — un interesado que pregunta por un inmueble desde el aviso. opt_out es definitivo hasta que la persona vuelva a escribir: si alguien responde «STOP», seguir escribiéndole por plantilla es la forma más rápida de perder el número.

comunicados FASE 2

El aviso en sí: qué se dice, a quién alcanza y por dónde sale. Un comunicado se redacta una vez y se entrega por varios canales.

CAMPOS	<code>id</code> , <code>autor_id</code> , <code>ambito</code> enum <code>unidad·contrato·portafolio·edificacion·global</code> , <code>inmueble_id</code> NULL, <code>contrato_id</code> NULL, <code>propietario_id</code> NULL, <code>tipo</code> enum <code>aviso·mantenimiento·incremento·canon·recordatorio·emergencia·normativo·comercial</code> , <code>titulo</code> , <code>cuerpo</code> TEXT, <code>prioridad</code> enum <code>baja·normal·alta·urgente</code> , <code>requiere_confirmacion</code> BOOL, <code>canales</code> JSON, <code>plantilla_id</code> NULL, <code>adjunto_archivo_id</code> NULL, <code>estado</code> enum <code>borrador·programado·enviando·enviado·cancelado</code> , <code>programado_para</code> , <code>enviado_at</code> , <code>created_at</code> , <code>updated_at</code>
ÍNDICES	<code>IDX(ambito, inmueble_id)</code> · <code>IDX(estado, programado_para)</code> — la cola de envío
NOTAS	<code>ambito</code> resuelve el alcance sin cuatro tablas: <code>unidad</code> le llega a quien vive ahí, <code>portafolio</code> a los inquilinos de todas las unidades de ese propietario, <code>edificacion</code> lo manda el administrador del edificio a todos sus residentes, <code>global</code> lo manda Yalqui. <code>requiere_confirmacion</code> existe para los avisos que legalmente conviene poder probar que se leyeron — un incremento de canon, sobre todo — y es lo que convierte el comunicado en evidencia y no en un mensaje más.

comunicado_destinatarios FASE 2

Quién recibió cada comunicado y qué hizo con él. Es la mitad que convierte un envío en una constancia.

CAMPOS	<code>id</code> , <code>comunicado_id</code> , <code>usuario_id</code> , <code>rol_destinatario</code> enum <code>inquilino·propietario·garante·proveedor</code> , <code>estado</code> enum <code>pendiente·enviado·entregado·leido·confirmado·fallido</code> , <code>leido_at</code> , <code>confirmado_at</code>
ÍNDICES	<code>UNIQUE(comunicado_id, usuario_id)</code> · <code>IDX(usuario_id, estado)</code>
NOTAS	Se resuelve al momento de enviar y se congela: si mañana el inquilino se muda, el comunicado que recibió sigue siendo suyo. El envío concreto por cada canal vive en mensajes con <code>origen_tipo = comunicado</code> ; aquí queda el estado consolidado de la persona.

preferencias_notificacion FASE 2

Qué acepta recibir cada usuario, por categoría y por canal. La primera compuerta de todo envío.

CAMPOS	<code>id</code> , <code>usuario_id</code> , <code>categoria</code> enum <code>cobranza·comunicado·incidencia·contrato·score·vecinos·comercial</code> , <code>canal</code> enum <code>whatsapp·email·sms·push·app</code> , <code>habilitado</code> BOOL, <code>updated_at</code>
ÍNDICES	<code>UNIQUE(usuario_id, categoria, canal)</code>
NOTAS	Se separa categoría de canal a propósito: querer los avisos de mora por WhatsApp pero las novedades comerciales por ningún lado es una preferencia legítima y frecuente. La categoría <code>edificacion</code> gobierna las peticiones de ayuda de vecinos: es la que hay que poder apagar sin apagar nada más, porque no todo el mundo quiere que le escriban a las 11 de la noche por unas llaves. Las categorías <code>cobranza</code> y <code>contrato</code> no se pueden desactivar del todo — hay obligación de notificar — pero sí se puede elegir el canal, y eso debe quedar explícito en la interfaz en vez de aparecer como un interruptor que no responde.

Pedir Ayuda a Vecino

Un botón en la app del inquilino que publica una petición a los vecinos verificados de su edificación: se me quedaron las llaves, ¿alguien puede recibir un domicilio?, hay agua saliendo del apartamento de al lado. Es la función más distintiva del producto y también la que más cuidado necesita, porque difundir «estoy sola y no puedo entrar» junto con una dirección exacta es un riesgo real.

El modelo lo maneja con cuatro reglas, todas apoyadas en tablas que ya existen.

01 Solo alcanza a vecinos verificados de la misma edificación

El público de una solicitud son los usuarios con contrato vigente en unidades de la misma edificación_id, o dentro de un radio si la edificación es de tipo zona. No es un tablón abierto: alguien sin contrato vigente no ve ni puede responder.

02 La ubicación exacta no se revela hasta que hay acuerdo

La solicitud viaja con la edificación y el piso o la torre, nunca con el número de la unidad. El dato preciso se entrega cuando el solicitante **acepta** una oferta concreta — de ahí que ayuda_respuestas tenga un estado aceptada separado de ofrecida.

03 Emergencia no es lo mismo que favor

Con `urgencia = emergencia` la solicitud además notifica al propietario y a Yalqui, no solo a los vecinos. Un escape de gas no se puede quedar esperando a que alguien de la torre abra la app.

04 Ayudar suma al score, pero con freno

Una ayuda completada y confirmada por quien la pidió genera un evento positivo en la dimensión convivencia. Es el contrapeso exacto de las PQRS, que solo restan. Pero como un score portable con recompensas invita a inflarlo entre conocidos, el evento tiene tope por periodo y solo cuenta entre usuarios sin contrato en común.

solicitudes_ayuda FASE 3

La petición publicada a la edificación, con su alcance y su vencimiento.

CAMPOS	<code>id</code> , <code>solicitante_id</code> , <code>inmueble_id</code> , <code>edificacion_id</code> , <code>contrato_id</code> , <code>categoria</code> enum <code>acceso·prestamo·recibir_paquete·mascota·acompanamiento·emergencia·informacion·otro</code> , <code>urgencia</code> enum <code>baja·media·alta·emergencia</code> , <code>titulo</code> , <code>descripcion</code> TEXT, <code>radio_metros</code> NULL, <code>estado</code> enum <code>abierta·con_ofertas·aceptada·resuelta·expirada·cancelada</code> , <code>aceptada_respuesta_id</code> NULL, <code>confirmada_por_solicitante</code> BOOL, <code>expira_at</code> , <code>resuelta_at</code> , <code>created_at</code>
ÍNDICES	<code>IDX(edificacion_id, estado, created_at)</code> — el feed de vecinos · <code>IDX(solicitante_id, created_at)</code> · <code>IDX(estado, expira_at)</code>
NOTAS	<code>expira_at</code> es obligatorio y corto: una petición de ayuda vieja no es una petición, es ruido. <code>confirmada_por_solicitante</code> es lo único que habilita el evento de score — sin esa confirmación, ayudar no puntúa. <code>radio_metros</code> solo aplica cuando la edificación es de tipo zona.

ayuda_respuestas FASE 3

Los vecinos que se ofrecen. Pueden ser varios; el solicitante elige uno.

CAMPOS	<code>id</code> , <code>solicitud_id</code> , <code>vecino_id</code> , <code>mensaje</code> , <code>estado</code> enum <code>ofrecida·aceptada·declinada·retirada·completada</code> , <code>ofrecida_at</code> , <code>aceptada_at</code> , <code>completada_at</code>
ÍNDICES	<code>UNIQUE(solicitud_id, vecino_id)</code> — una oferta por vecino · <code>IDX(vecino_id, estado)</code>
NOTAS	El paso de <code>aceptada</code> a <code>completada</code> lo marca el vecino, pero solo se vuelve evento de score cuando el solicitante confirma en la solicitud. Dos manos para un mismo hecho: es lo mínimo cuando lo que está en juego es un puntaje que viaja con la persona.

archivos FASE 1

Registro único de todo lo que se sube a S3. Las demás tablas guardan `archivo_id`, nunca una URL.

CAMPOS	<code>id</code> , <code>uuid</code> , <code>s3_key</code> , <code>bucket</code> , <code>nombre_original</code> , <code>mime</code> , <code>tamano_bytes</code> BIGINT, <code>subido_por_id</code> , <code>entidad_tipo</code> , <code>entidad_id</code> , <code>publico</code> , <code>created_at</code>
NOTAS	Centralizarlo permite un solo lugar para URLs firmadas, cuotas por plan y borrado en cascada al eliminar una cuenta.

eventos_webhook FASE 2

Todo lo que entra desde afuera: resultados de Truora y centrales, confirmaciones de entrega de WhatsApp, avisos de la pasarela de pago.

CAMPOS	<code>id</code> , <code>proveedor</code> , <code>id_evento_externo</code> , <code>tipo</code> , <code>cuerpo</code> JSON, <code>firma_valida</code> BOOL, <code>procesado_at</code> , <code>error</code> , <code>created_at</code>
ÍNDICES	UNIQUE(<code>proveedor</code> , <code>id_evento_externo</code>) – los webhooks se reenvían · IDX(<code>procesado_at</code>)
NOTAS	Una sola tabla para los cuatro casos en vez de una por integración. Deduplica por el id del evento y no del recurso, porque un mismo <i>check</i> genera varios eventos legítimos. Y permite reprocesar sin pedirle al proveedor que reenvíe, que es lo que salva una tarde cuando un adaptador tenía un error. Ver el contrato de verificación .

auditoria FASE 2

Quién cambió qué y cuándo. Obligatoria sobre contratos, firmas, pagos y cambios de plan.

CAMPOS	<code>id</code> , <code>actor_id</code> NULL, <code>accion</code> , <code>entidad_tipo</code> , <code>entidad_id</code> , <code>antes</code> JSON, <code>despues</code> JSON, <code>ip</code> , <code>user_agent</code> , <code>created_at</code>
ÍNDICES	IDX(<code>entidad_tipo</code> , <code>entidad_id</code> , <code>created_at</code>)

app_meta FASE 1

Sobrevive al placeholder, con un propósito real: versión del esquema y banderas de configuración.

CAMPOS	<code>id</code> , <code>key</code> , <code>value</code> , <code>updated_at</code> – sin cambios respecto de hoy
NOTAS	Guardar ahí <code>schema_version</code> le da a <code>yalqui-init-db</code> algo contra qué comparar antes de aplicar el SQL. Alternativa: borrarla y migrar al patrón de migraciones versionadas de FRUBA. Recomendando conservarla.

Extienden lo que ya hace `app_meta`: `snake_case` en SQL, `camelCase` en el export de Drizzle, `int` autoincremental como llave, `timestamp` de MySQL para las fechas.

ASPECTO	REGLA	POR QUÉ
Nombres de tabla	snake_case, plural, español	Continúa app_meta; el dominio es colombiano y el equipo piensa en español
Llave primaria	INT UNSIGNED AUTO_INCREMENT; BIGINT en pagos, eventos y auditoría	Se mantiene el patrón de app_meta. BIGINT solo donde el volumen lo justifica
Identificador público	uuid CHAR(36) solo en inmuebles y contratos	La llave sigue siendo autoincremental como en app_meta. El UUID va únicamente donde la URL es pública o adivinable: la ficha del inmueble y el link de firma del contrato
Dinero	DECIMAL(14, 2), nunca FLOAT	Un canon de \$2.500.000 con error de redondeo es una demanda
Moneda	CHAR(3) con default COP	Cuesta nada hoy y evita una migración si Yalqui sale del país
Fechas	DATE para lo contractual, TIMESTAMP para lo del sistema	La fecha de inicio del contrato no tiene hora ni zona horaria
Estados	ENUM de MySQL vía mysqlEnum	Conjuntos cerrados que solo cambian con código nuevo; el motor los valida
Catálogos	Tablas con seeders: planes, servicios, amenidades	Cambian por decisión comercial, no por despliegue
Cadenas indexadas	VARCHAR(191)	Mismo límite que usa app_meta hoy
Timestamps	created_at + updated_at en toda tabla mutable	app_meta solo tiene updated_at; se agrega el par completo
Borrado	estado o archivado_at; nunca DELETE en contratos, facturas o pagos	Son documentos con valor legal y contable
FK huérfanas	ON DELETE RESTRICT por defecto; CASCADE solo en tablas hijas puras	Fotos y conceptos sí caen con su padre; un contrato jamás

Un archivo por módulo, no un **schema.ts** de mil líneas. Propongo partir backend/src/db/schema.ts en schema/identidad.ts, schema/permisos.ts, schema/inmuebles.ts, schema/ajustes.ts, schema/contratos.ts, schema/arriendo.ts, schema/suscripciones.ts, schema/operacion.ts, schema/scoring.ts, schema/comunicacion.ts, schema/edificación.ts, y dejar schema.ts como barril que reexporta todo. backend/src/db/index.ts no cambia — sigue haciendo import * as schema. El schema.sql sí queda como un solo archivo, ordenado por dependencias de FK, porque init-handler.ts lo aplica de una sola pasada con multipleStatements.

05 Fases de construcción

Ochenta y siete tablas de golpe no se pueden probar. El corte por fases busca que al final de cada una haya un recorrido completo funcionando de punta a punta.

FASE	QUÉ HABILITA	TABLAS	TOTAL
FASE 1	Recorrido mínimo real: registrar la unidad con sus dueños y su canon configurable, agendar visita, aplicar, firmar con todos los del título, facturar con desglose y reportar el primer pago	usuarios, usuario_rols, perfiles_propietario, perfiles_inquilino, archivos, inmuebles, inmueble_fotos, catalogo_ajustes, inmueble_ajustes, contrato_ajustes, visitas, aplicaciones, contratos, contrato_firmas, facturas_arriendo, factura_arriendo_conceptos, pagos_arriendo, planes, suscripciones, consentimientos, mensajes, inmueble_propietarios, aplicacion_ajustes, aplicacion_documentos, plantillas_contrato, etiquetas, inmueble_etiquetas, precalificaciones, precalificacion_aportantes, reglas_precalificacion, app_meta	31
FASE 2	Lo que le quita trabajo al propietario con varias unidades: verificación en lote, incremento anual, salida del inquilino, sus propias obligaciones. Más screening, score, comunicados y personal del edificio	proveedores, valoraciones, verificaciones, garantes, contrato_anexos, valoracion_ajustes, reglas_verificacion_pago, incrementos_canon, terminaciones, inspecciones, obligaciones_propietario, descuentos_volumen, servicios, servicios_contratados, facturas_yalqui, factura_yalqui_conceptos, pagos_yalqui, incidencias, incidencia_eventos, alertas, auditoria, plan_caracteristicas, dimensiones_score, eventos_score, scores_inquilino, score_dimensiones, plantillas_mensaje, conversaciones_whatsapp, comunicados, comunicado_destinatarios, preferencias_notificacion, empresas_servicio_publico, cuentas_cobro, edificaciones, proveedores_verificacion, aplicacion_verificaciones, personal_propiedad, permisos_rol, eventos_webhook, canales_publicacion, publicaciones_externas, analisis_ia	42
FASE 3	Sofisticación: conciliación bancaria, expediente de mora, rentabilidad por unidad, servicios públicos por API, convivencia con evidencia	movimientos (unidad y edificación), movimientos_bancarios, expedientes_mora, publicacion_metricas, facturas_cobro, solicitudes_ayuda, ayuda_respuestas, pqrs, inquilino_atributos, negociaciones, negociacion_ofertas, cotizaciones, amenidades, inmueble_amenidades	14

NOTA El `init-handler.ts` aplica el SQL completo con `CREATE TABLE IF NOT EXISTS`, así que cada fase se puede agregar al mismo archivo sin romper lo ya desplegado — mientras no haya que alterar columnas existentes. En cuanto aparezca el primer `ALTER`, toca migraciones versionadas de verdad.

06 Lo que no se modela todavía — y dónde se engancha

Las integraciones futuras no llevan tablas propias por ahora, pero el modelo ya deja el punto donde cada una entra sin rediseño.

INTEGRACIÓN	PUNTO DE ENGANCHE EXISTENTE
Inmobiliarias y administradores	Fuera de alcance por decisión de producto , no por limitación técnica. Si algún día entran, el cambio es una tabla organizaciones más una migración de <code>propietario_id</code> a <code>organizacion_id</code> en tres tablas. Se deja anotado, no preparado: columnas nulas «por si acaso» envejecen peor que una migración honesta.
Portales y redes	Ya modelado en M2 . Página de Facebook, Instagram y catálogo <code>home_listing</code> se automatizan sin permisos especiales; Marketplace no — su fila nace <code>no_disponible</code> hasta que exista alianza con Meta.
Pasarela de link de pago	<code>facturas_arriendo.url_pago</code> y <code>pagos_arriendo.referencia_externa</code> ; el webhook lleva el pago directo a verificado sin intervención del propietario.
Empresas de servicios públicos	Ya modelado en M2 , no pendiente. <code>empresas_servicio_publico.metodo_consulta</code> asume que la mayoría no tendrá API: se arranca con comprobante subido por el inquilino y se automatiza empresa por empresa. Lo que no cambia es el esquema.
Bancos	<code>pagos_arriendo</code> ya guarda <code>banco_origen</code> , <code>referencia_externa</code> y <code>fecha_pago_declarada</code> : es exactamente lo que hace falta para cruzar contra un extracto y pasar la verificación de manual a automática.
DIAN	<code>facturas_yalqui.cufe</code> y <code>perfiles_propietario.responsable_iva</code> están desde la fase 2.
Truora y centrales de riesgo	Ya modelado en M4 , no pendiente. <code>proveedores_verificacion</code> enruta por tipo y guarda costo y vigencia; <code>verificaciones</code> maneja el ciclo asíncrono con <code>id_externo</code> y <code>webhook</code> .
Administración de copropiedades	<code>pqrs.origen = administracion</code> ya contempla que la queja llegue de un sistema de administración de edificios y no de una persona.
Firma electrónica certificada	<code>contrato_firmas.proveedor_firma + evidencia</code> JSON; hoy sirve para firma propia con OTP.
WhatsApp Business API	Todo el módulo M11 está construido alrededor de sus reglas: <code>plantillas_mensaje</code> guarda el nombre y el estado de aprobación en Meta, <code>conversaciones_whatsapp</code> lleva la ventana de 24 horas, y <code>mensajes</code> guarda el <code>id_externo</code> para reconciliar los webhooks de entrega y lectura.

Cerrado y pendiente

Decisiones cerradas

DECISIÓN	QUÉ SE RESOLVIÓ	QUÉ CAMBIÓ EN EL MODELO
Recaudo	Yalqui no recauda. El canon va directo al propietario; Yalqui puede poner el link de pago y guarda el comprobante	Se elimina dispersiones. pagos_arriendo pasa de transacción a comprobante con ciclo de verificación
Planes	Manda el brief: Básico gratis, Profesional, Empresarial, Corporativo	Seeders de planes. Se caen min_unidades y max_unidades
Unidad	La unidad arrendable es lo facturable y sobre lo que se paga arriendo	Una fila de inmuebles por unidad. La agrupación se resolvió después con etiquetas, no con jerarquía
Suscripción	Por inmueble, no por propietario. Un propietario tiene varias	suscripciones.inmueble_id obligatorio, sin unidades_contratadas. La factura mensual agrupa por propietario con un concepto por unidad
Roles	Siete: administrador Yalqui, administrador de inmueble, propietario, socio propietario, inquilino, personal de propiedad y proveedor	usuario_rols gana ámbito (global, inmueble, edificación o contrato), porque «es propietario» no dice de qué. Aparecen inmueble_propietarios, personal_propiedad y permisos_rol. El administrador de inmueble solo lo otorga a mano un admin de Yalqui, contra acta de asamblea
Copropiedad	Una unidad puede tener varios dueños	inmuebles.propietario_id pasa a significar «el principal» y no cambia de tipo. Los socios y sus porcentajes van aparte. Los que están en el título firman el contrato
Segmento	El cliente es el propietario individual, no la inmobiliaria ni el administrador de portafolio ajeno	No hay organizaciones ni membresías. propietario_id apunta a usuarios en todo el esquema. Los cuatro planes se diferencian por funciones, no por tamaño de portafolio — sube plan_caracteristicas a fase 2
Score portable	El puntaje viaja con el inquilino entre propietarios	Aparece consentimientos. El puntaje y su desglose se comparten; los eventos no, porque contienen datos de otros propietarios. eventos_score gana caducidad y ciclo de reclamo
Servicios a la carta	Se cobran por unidad, igual que la suscripción	servicios_contratados.inmueble_id pasa a obligatorio. No existe el servicio de cuenta: Legal para ocho unidades son ocho líneas en la factura
Screening externo	Truora para identidad y antecedentes, centrales de riesgo para crédito	proveedores_verificacion como catálogo. Las verificaciones pasan a colgar de la persona y no de la aplicación , con vigente_hasta: aplicar a cinco inmuebles ya no cuesta cinco consultas
Ayuda entre vecinos	Botón del inquilino para pedir ayuda a su edificación	Aparece edificaciones, que es vecindad verificable y no un rótulo cualquiera, más solicitudes_ayuda y ayuda_respuestas. Ayudar suma a la convivencia, con confirmación de dos partes y tope
Comprobantes de lo que va aparte	El inquilino ve cada mes qué le toca pagar por fuera del canon — servicios y administración cuando aplique —, sube el comprobante y lo marca	Las tablas se renombran a cuentas_cobro y facturas_cobro: ya no son solo servicios públicos. Mismo ciclo de dos manos que el canon, reportada → verificada. La cuota extraordinaria va aparte y por defecto la paga el propietario
Trabajo del propietario	Revisión desde el lado de quien tiene ocho unidades: diez ajustes al sistema	Verificación en lote y reglas · conciliación bancaria · bandeja de WhatsApp entrante por unidad · incremento anual del IPC con notificación · campos de IVA y retención · flujo de salida con días vacía · calendario de obligaciones del propietario · expediente de mora · garantía según marco legal · comparar aplicantes · descuento por volumen
Precalificación	En la visita se manda un enlace, el interesado declara ingresos y el sistema verifica que el canon no pase del tope	Aparecen precalificaciones y precalificacion_aportantes. El tope vive en inmuebles.tope_ingreso_pct, 50% por defecto. Preaprobado es sobre ingresos declarados ; los documentos vienen después
Crear edificación	Se puede crear una edificación y asociarle unidades, o registrar una unidad suelta sin edificación	inmuebles.edificacion_id nulo. Los dos caminos son de primera clase; asociar después es llenar una columna, no migrar

DECISIÓN	QUÉ SE RESOLVIÓ	QUÉ CAMBIÓ EN EL MODELO
Edificación propia	Un propietario puede ser dueño de toda la edificación y arrendar las unidades por separado	edificaciones gana regimen: en propiedad_unica las zonas comunes son del dueño y él es administrador sin otorgamiento manual. movimientos pasa a dos niveles con prorrateo, para que pintar la fachada no haya que inventarle una unidad
Etiquetas	Una unidad no pertenece a otra. El portafolio es plano y se organiza con etiquetas, filtrables y agrupables	Se cae inmuebles.inmueble_padre_id. Aparecen etiquetas e inmueble_etiquetas, con es_principal para que agrupar no duplique unidades ni infle los totales. edificacion_id se queda: el edificio es un hecho, la etiqueta es una opinión
Tipos de unidad	Apartamento, casa, local, oficina, habitación, parqueadero (más bodega y lote)	Enum ampliado en inmuebles.tipo. El parqueadero puede ser unidad propia o ajuste al canon de otra — el modelo admite las dos
Canon configurable	Valor base más opciones que lo ajustan: amoblado, persona adicional, mascota, parqueadero, servicios	Tres tablas: catalogo_ajustes define qué existe, inmueble_ajustes qué ofrece la unidad y a cuánto, contrato_ajustes qué se pactó y congela el precio. La factura los desglosa línea por línea
Servicios públicos	Cada unidad detalla si van incluidos, y el propietario puede ver si están pagos	cuentas_cobro con incluido_en_canon y tope_incluido; facturas_cobro con estado por periodo y desconocido como estado válido
Precios	Por unidad y por mes: Básico \$0, Profesional \$20.000, Empresarial \$25.000, Corporativo \$30.000	Seeder de planes resuelto
Básico y servicios	Un propietario en el plan gratuito sí puede comprar servicios a la carta	Sin validación de servicios_contratados contra el plan. Una unidad gratuita puede generar factura de Yalqui
Comunicados	Sección propia, con salida a WhatsApp	Módulo M11 con seis tablas. gestiones_cobranza desaparece: se absorbe en mensajes, el registro único de salida
Reglas del score	Delegadas a criterio técnico	Seis reglas en M10: sin datos demográficos, con evidencia rastreable, con descargos, con caducidad, con reclamo y explicación en lenguaje llano, y distinguiendo «sin historia» de «mal puntaje»
Transparencia	Estándar Nubank: nada opaco	El esquema guarda la explicación, no solo el dato — explicacion_publica en eventos y dimensiones, delta en el score, alerta score_actualizado al inquilino y no solo al propietario
Llaves	Se mantiene autoincremental, como app_meta	uuid queda solo en inmuebles y contratos, donde la URL es pública

Lo único que falta

Ninguna decisión de diseño. Un dato de producto, y no bloquea la fase 1.

01 Qué desbloquea cada plan

Con los precios cerrados queda la otra mitad: la escalera va \$0 → \$20.000 → \$25.000 → \$30.000 por unidad al mes, y hoy nada en el modelo dice en qué se diferencian. Como el cobro es por unidad y el cliente es un propietario individual, los planes no se pueden separar por tamaño de portafolio ni por número de usuarios — solo por **qué habilita cada uno en esa unidad**: cuántas fotos, si trae cobranza automática, si incluye screening, cuántos comunicados por WhatsApp al mes, si da reportes de rentabilidad.

Ese cuadro es el contenido de plan_caracteristicas. La tabla ya está diseñada y entra en fase 2, así que la fase 1 corre sin él — pero conviene tenerlo antes de publicar precios, porque entre \$25.000 y \$30.000 hay \$5.000 que hoy no compran nada distinto.

DATO ÚTIL Los comunicados por WhatsApp tienen costo real por conversación iniciada. Es un buen candidato a característica escalonada por plan: es lo único del producto donde un tope tiene una justificación económica evidente y no arbitraria.

El diseño está cerrado y no queda ninguna decisión pendiente. **Falta tu visto bueno para escribir.**

Cuando lo des, la fase 1 son diecinueve tablas: `backend/src/db/schema/` partido por módulo con un barril que reexporta, `backend/sql/schema.sql` ordenado por dependencias de llave foránea, y los seeders de planes con los cuatro precios ya definidos. De ahí se invoca `yalqui-init-db` contra RDS y se ve el modelo en pie.

La fase 1 deja el recorrido completo funcionando: registrar la unidad, agendar la visita, aplicar, firmar, facturar el canon, reportar el pago con su comprobante y avisar por WhatsApp. Las plantillas de Meta y los comunicados entran en la fase 2, cuando ya haya a quién escribirle.